

This presentation uses the Pretendard and D2Coding font.

.NET Universe 2026

Korea's largest .NET developer conference

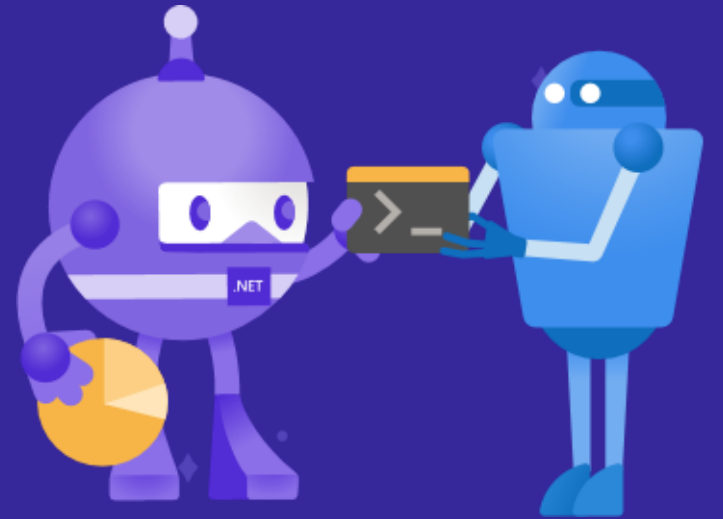
Presented by



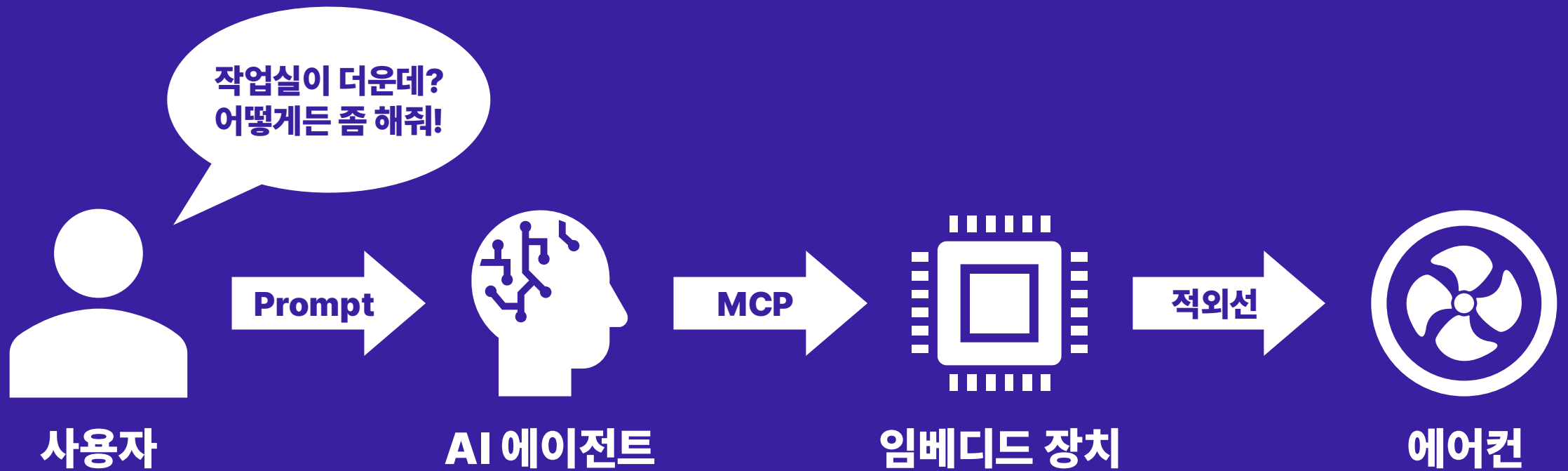
**.Net
Dev**

.NET nanoFramework로 MCP 서버 임베디드 장치 만들기

김준형



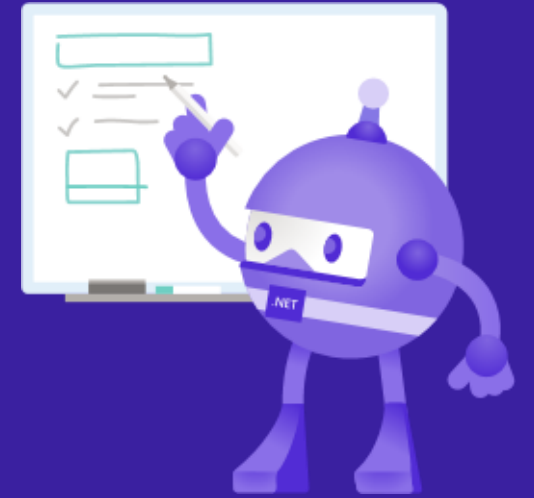
정확히 무엇을 만들었나?



목차

- **.NET nanoFramework 소개**
- **개발 환경 준비하기**
- **에어컨 리모컨 만들기**
- **Web API 기능 추가**
- **MCP 서버 기능 추가**
- **Demonstration**





.NET nanoFramework 소개

MCU란?

The image shows a Google search results page for the query 'MCU'. The browser's address bar shows the search URL. The search results include entries from NamuWiki, Naver Blog, and Wikipedia. A red line is drawn under the NamuWiki title '마블 시네마틱 유니버스'. A red arrow points from a small suggestion box on the right, which contains the text '이것을 찾으셨나요?' and '마이크로컨트롤러 컴퓨터 프로세서', to a larger, more detailed box on the right side of the image. This larger box also contains the text '마이크로컨트롤러 컴퓨터 프로세서' next to a small image of a microcontroller chip.

MCU - Google 검색

https://www.google.com/search?q=MCU&scasv=ae999b120404273d&source=hp&lei=NzylaZy1EbSi0-kPrJLqiAY&uifsig=AfdpzrgAAAAAaVKRwwmtW...

Google MCU

AI 모드 전체 이미지 뉴스 동영상 쇼핑 짧은 동영상 더보기 도구

나무위키
https://namu.wiki > ...
마블 시네마틱 유니버스
8일 전 — 마블 스튜디오에서 제작하는 슈퍼히어로물 프랜차이즈 세계관 Marvel Cinematic Universe (마블 시네마틱 유니버스) 앞자를 따서 MCU라고 부른다.[2] ...

Naver Blog
https://blog.naver.com > techref > ...
MCU 란? 주요 특징과 종류(ARM, AVR, PIC, MSP, Tricore) ...
2023. 7. 17. — MCU는 마이크로컨트롤러 유닛의 약자로, 작은 규모의 컴퓨터 시스템을 하나의 칩 안에 통합한 것입니다. MCU는 중앙 처리 장치 (CPU), 메모리 (RAM 및 ROM) ...

Wikipedia
https://ko.wikipedia.org > wiki > 마블_시네마틱_유니... > ...
마블 시네마틱 유니버스 - 위키백과, 우리 모두의 백과사전
마블 시네마틱 유니버스(Marvel Cinematic Universe, MCU)는 마블 코믹스의 만화 작품에 기반하여, 마블 스튜디오가 제작하는 슈퍼히어로 영화를 중심으로 드라마, ...

이것을 찾으셨나요?
마이크로컨트롤러
컴퓨터 프로세서

마이크로컨트롤러
컴퓨터 프로세서

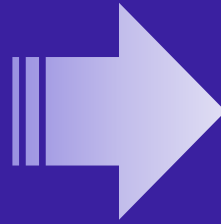
Micro Controller Unit

- 프로세서와 메모리, 입출력 모듈을 단일 칩에 내장
- 임베디드 시스템 구축에 널리 사용
- OS 없이 Bare-metal, 또는 RTOS로 멀티태스킹 기반 운영 가능
- 하드웨어 인터페이스를 위한 GPIO, I2C, SPI, UART 등 지원
- 프로그램 개발에는 주로 C, C++ 언어를 사용

그러면, C#으로는 개발이 불가능한가?

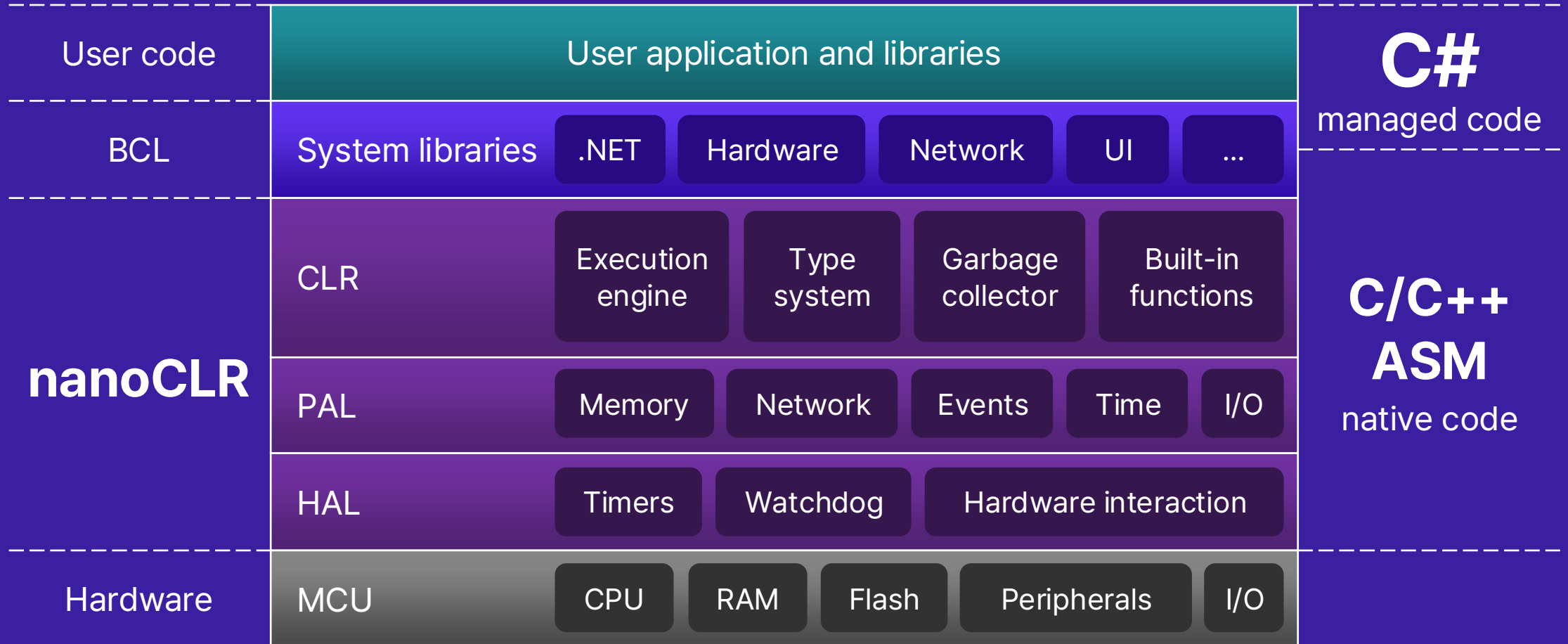


2007 ~ 2015

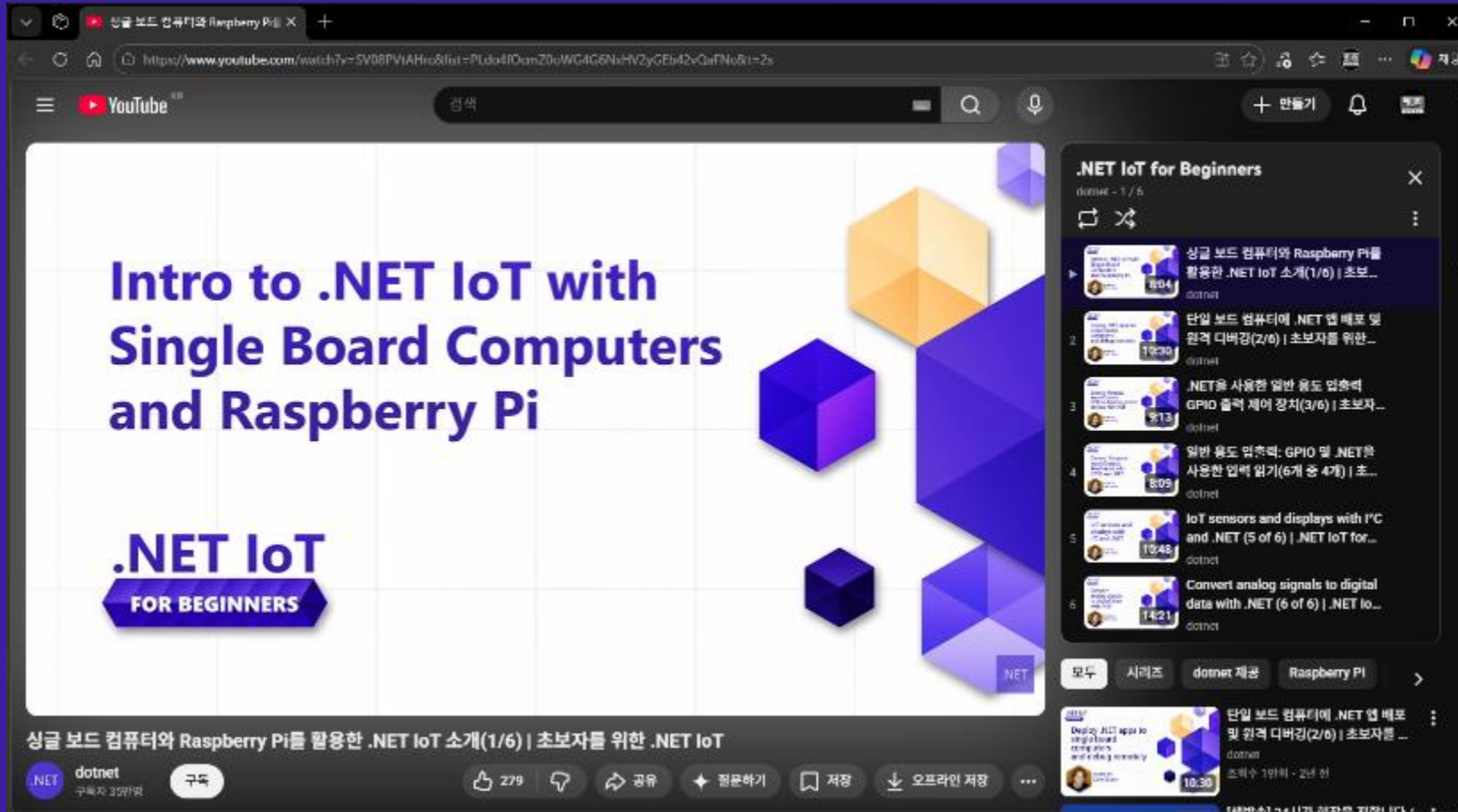


2017 ~ 현재

.NET nanoFramework Architecture



비슷하지만, 완전히 다른 .NET IoT



The screenshot shows a YouTube video player interface. The video title is "Intro to .NET IoT with Single Board Computers and Raspberry Pi". Below the title, it says ".NET IoT FOR BEGINNERS". The video player shows the first frame of the video, which has a white background with a grid pattern and several 3D cubes in orange and purple. The video player controls are visible at the bottom, showing 279 likes, a share button, a playlist button, a bookmark button, and a download button. The video is from the channel "dotnet" and has 3,977 views.

Intro to .NET IoT with Single Board Computers and Raspberry Pi

.NET IoT FOR BEGINNERS

싱글 보드 컴퓨터와 Raspberry Pi를 활용한 .NET IoT 소개(1/6) | 초보자를 위한 .NET IoT

dotnet 구독자 3,977명

279 좋아요

공유

playlist

저장

다운로드

오프라인 저장

.NET IoT for Beginners

dotnet - 1 / 6

- 1. 싱글 보드 컴퓨터와 Raspberry Pi를 활용한 .NET IoT 소개(1/6) | 초보... dotnet 8:04
- 2. 단일 보드 컴퓨터에 .NET 앱 배포 및 원격 디버깅(2/6) | 초보자를 위한... dotnet 19:30
- 3. .NET을 사용한 일반 용도 입출력: GPIO 출력 제어 장치(3/6) | 초보자... dotnet 3:13
- 4. 일반 용도 입출력: GPIO 및 .NET을 사용한 입력 읽기(6개 중 4개) | 초... dotnet 8:09
- 5. IoT sensors and displays with I²C and .NET (5 of 6) | .NET IoT for... dotnet 19:48
- 6. Convert analog signals to digital data with .NET (6 of 6) | .NET IoT... dotnet 14:21

모두 시리즈 dotnet 제공 Raspberry Pi

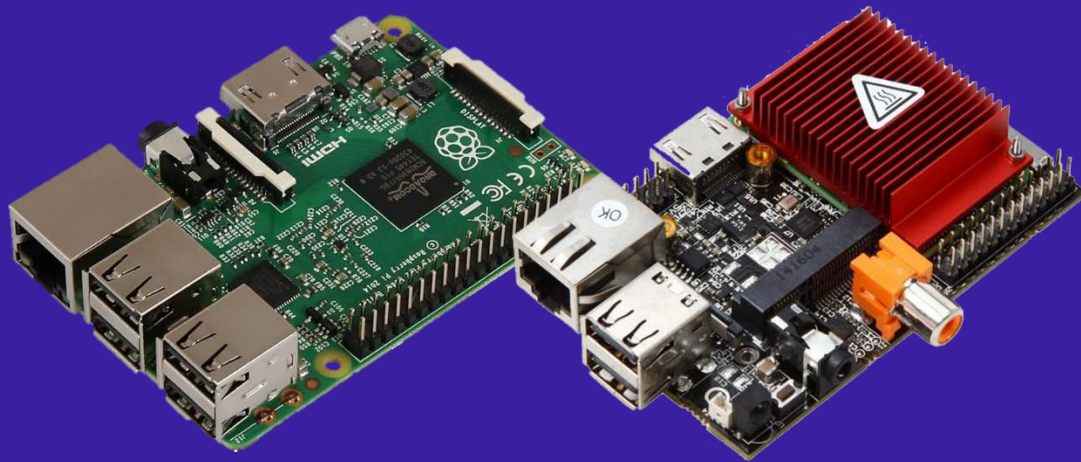
Deploy IoT apps to single-board computers and Raspberry Pi | dotnet 10:30

단일 보드 컴퓨터에 .NET 앱 배포 및 원격 디버깅(2/6) | 초보자를 위한... dotnet

조회수 1만회 · 2년 전

.NET IoT vs .NET nanoFramework

- SBC (Single Board Computer)
- GB 단위 RAM
- Linux, Windows IoT
- .NET 기반 라이브러리 사용 가능
- 복잡한 로직 및 디스플레이에 적합

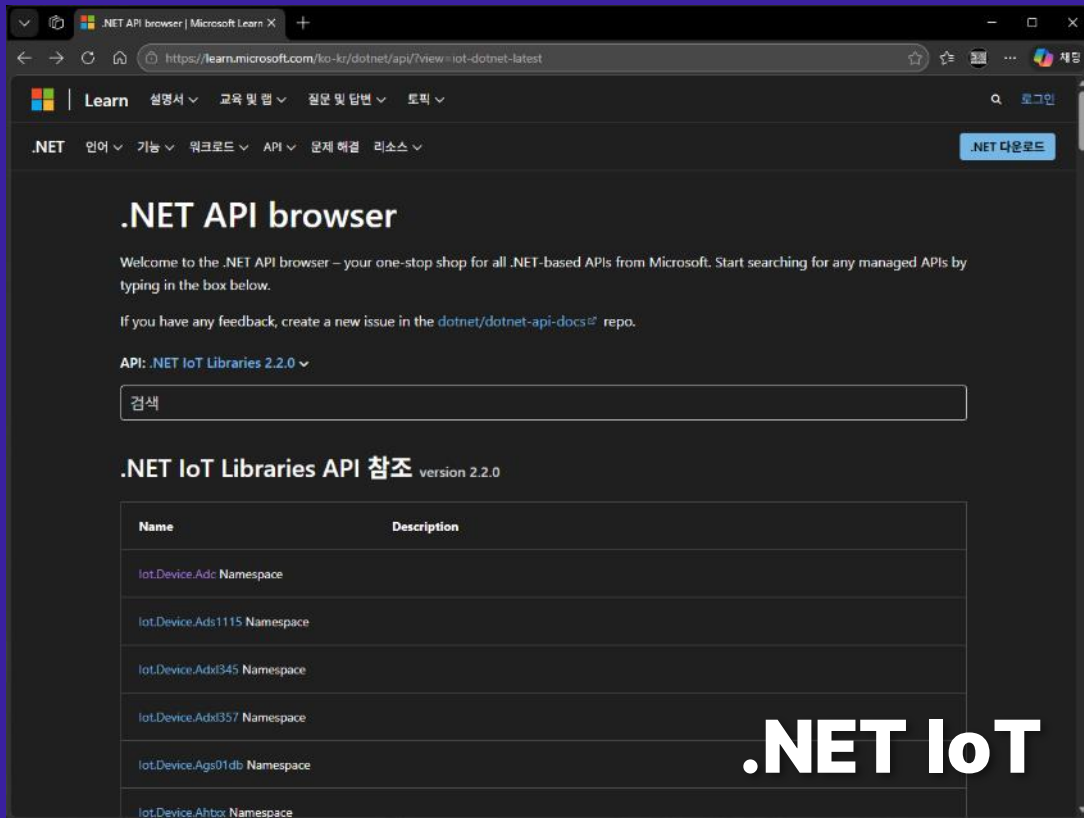


- MCU (Micro Controller Unit)
- MB 단위 RAM
- RTOS
- nfproj 기반 라이브러리만 사용 가능
- 저전력 센서 웨어러블에 적합



.NET IoT와 유사한 네임스페이스 구성

learn.microsoft.com/dotnet/api/?view=iot-dotnet-latest

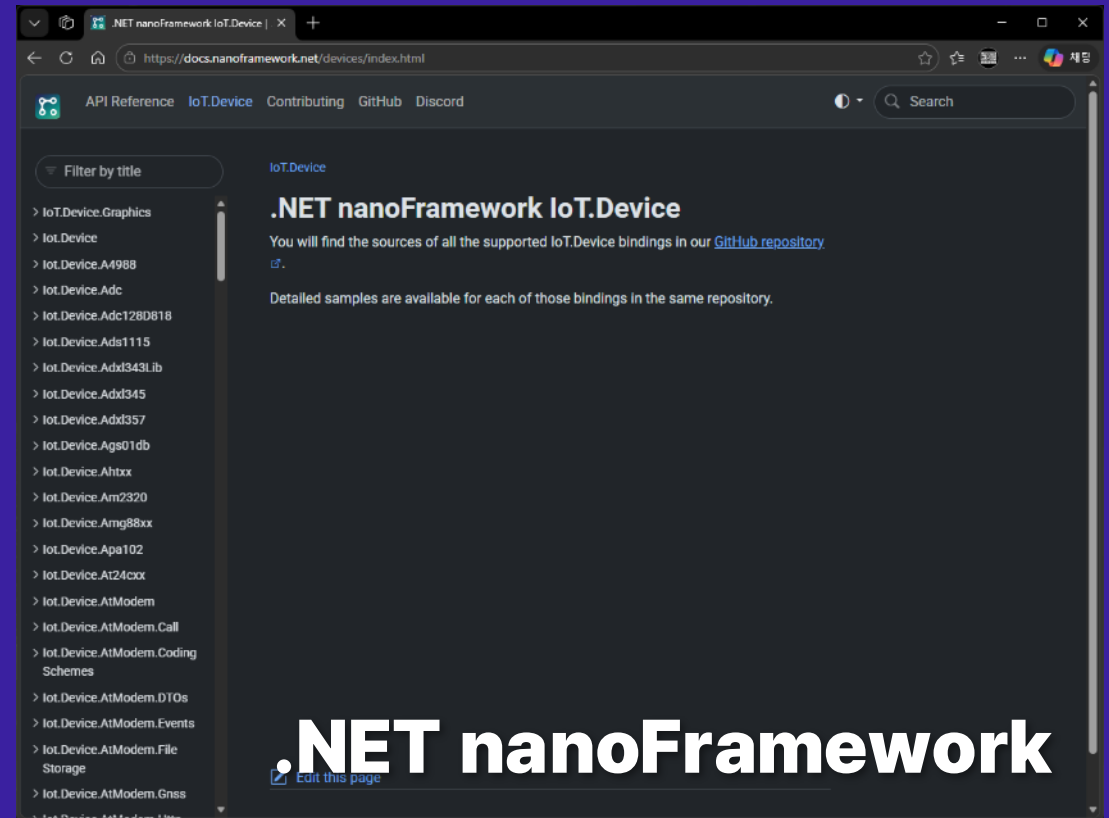


The screenshot shows the .NET API browser interface. The main heading is ".NET API browser". Below it, a welcome message states: "Welcome to the .NET API browser – your one-stop shop for all .NET-based APIs from Microsoft. Start searching for any managed APIs by typing in the box below." A search box with the placeholder text "검색" is visible. Below the search box, the text ".NET IoT Libraries API 참조 version 2.2.0" is displayed. A table with two columns, "Name" and "Description", lists several namespaces:

Name	Description
IoT.Device.Adc Namespace	
IoT.Device.Ads1115 Namespace	
IoT.Device.Adx345 Namespace	
IoT.Device.Adx357 Namespace	
IoT.Device.Ags01db Namespace	
IoT.Device.Ahtbox Namespace	

.NET IoT

docs.nanoframework.net/devices/index.html



The screenshot shows the .NET nanoFramework IoT.Device API reference website. The main heading is ".NET nanoFramework IoT.Device". Below it, a message states: "You will find the sources of all the supported IoT.Device bindings in our [GitHub repository](#)." A list of bindings is shown on the left, including IoT.Device.Graphics, IoT.Device, IoT.Device.A4988, IoT.Device.Adc, IoT.Device.Adc128D818, IoT.Device.Ads1115, IoT.Device.Adx343Lib, IoT.Device.Adx345, IoT.Device.Adx357, IoT.Device.Ags01db, IoT.Device.Ahtxx, IoT.Device.Am2320, IoT.Device.Amg88xx, IoT.Device.Apa102, IoT.Device.At24cxx, IoT.Device.AtModem, IoT.Device.AtModem.Call, IoT.Device.AtModem.Coding Schemes, IoT.Device.AtModem.DTOS, IoT.Device.AtModem.Events, IoT.Device.AtModem.File Storage, and IoT.Device.AtModem.Gnss.

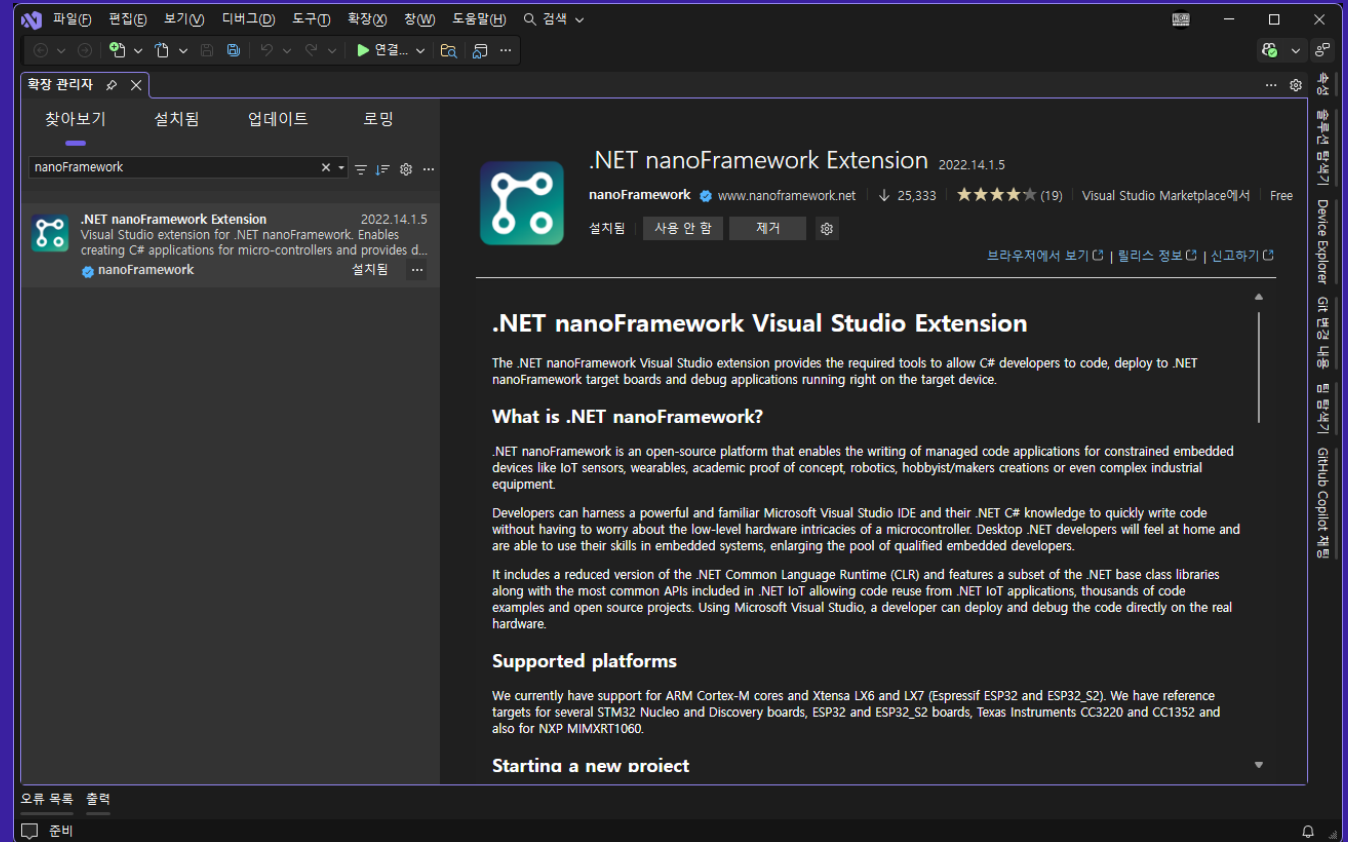
.NET nanoFramework

개발 환경 준비하기



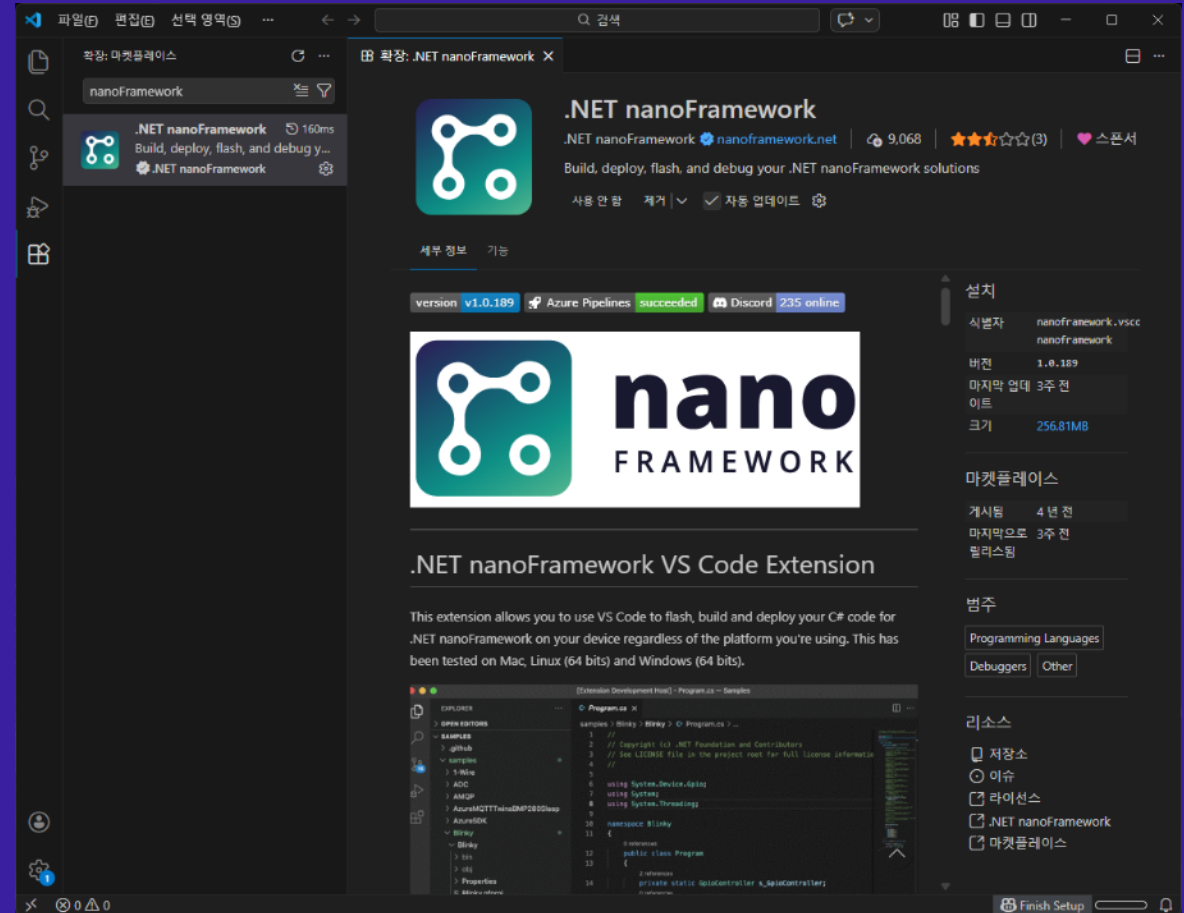
Visual Studio 2022 확장

- VS 2019, 2026 호환
- 프로젝트 파일: nfproj
- 실시간 디버깅 가능!



Visual Studio Code 확장

- Windows, Mac, Linux 지원
- x64 및 M1에서 개발 가능
- 추가 필요 사항
 - .NET 6.0
 - Visual Studio build tools
 - mono-complete (Linux/Mac)
- 32bit OS나 ARM은 사용 불가
- 실시간 디버깅 불가...



개발용 MCU 보드 선택

- Espressif Systems
 - ESP32, ESP32-S Series, ESP32-C Series
- STMicroelectronics
 - STM32 NUCLEO, F429, F769
- Texas Instruments
 - CC1352R1, CC3220SF
- NXP Semiconductors
 - MXRT1060



ESPRESSIF



life.augmented



TEXAS INSTRUMENTS

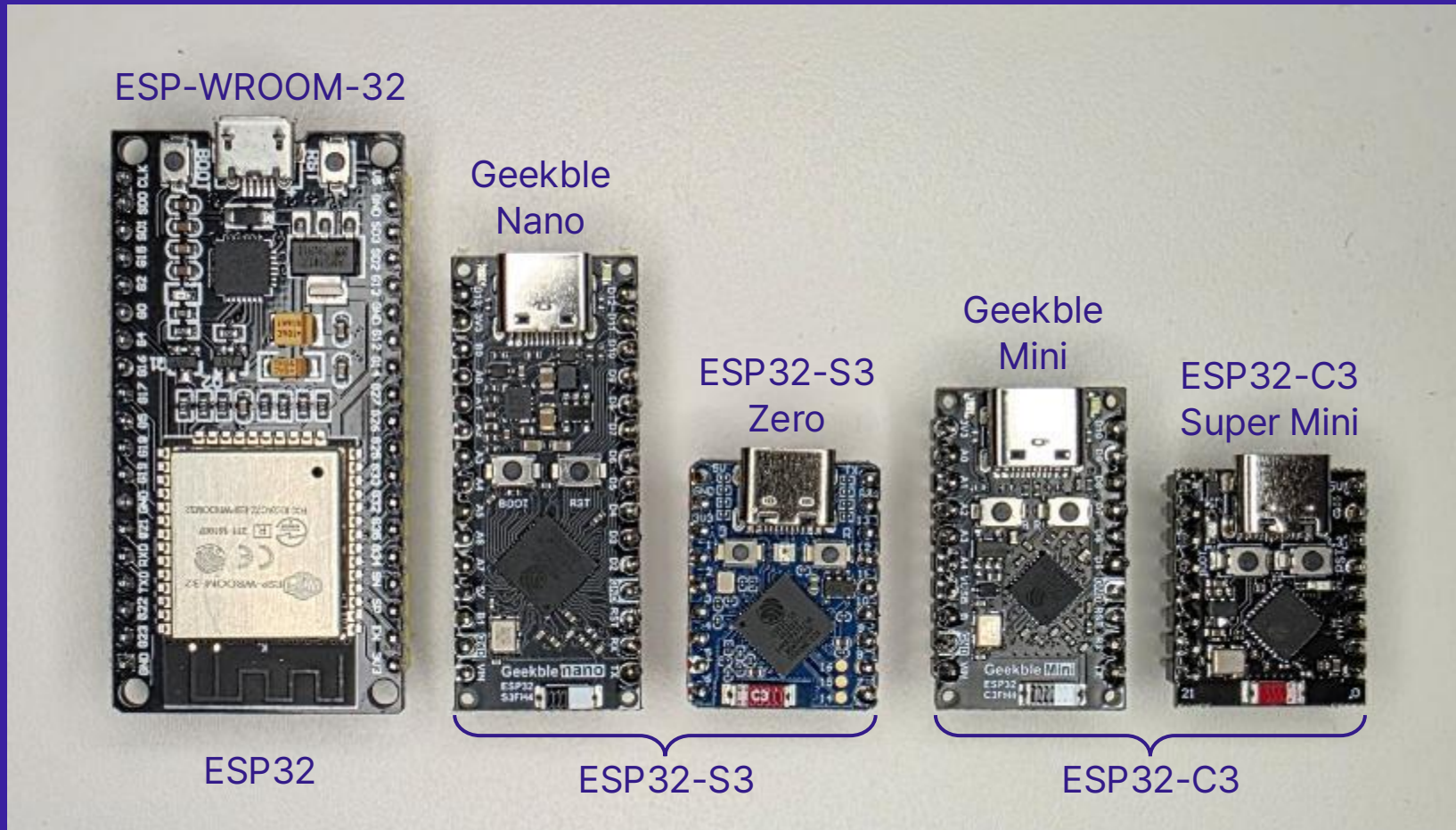


ESP32 시리즈

- Wi-Fi, Bluetooth를 자체적으로 지원

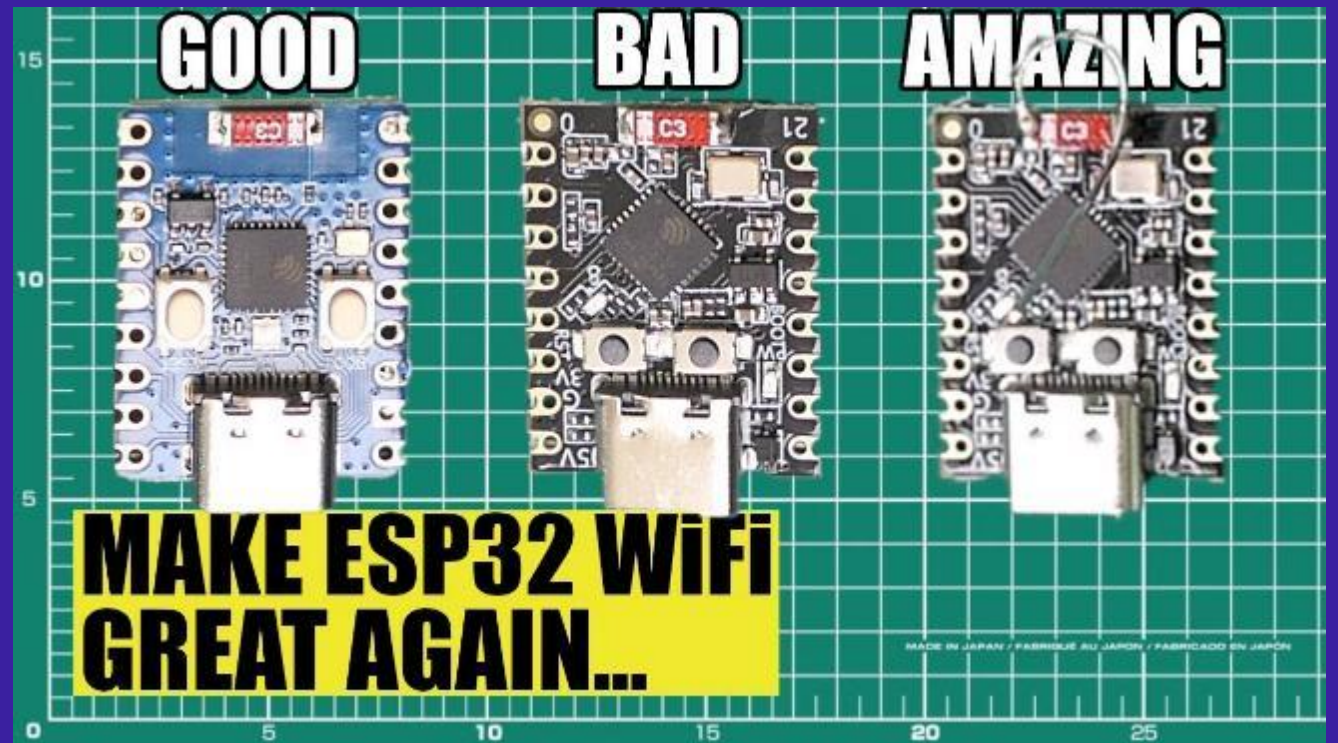
MCU	ESP32	ESP32-S3	ESP32-C3
특징	대표 모델, 안정된 라이브러리	고성능, SIMD 가속 지원	최저사양, 저전력 설계
프로세서	Xtensa® Dual-Core LX6	Xtensa® Dual-Core LX7	RISC-V Single-Core
최대 클럭	240 MHz		160 MHz
ROM	448 KB	384 KB	
SRAM	520 KB	512 KB	400 KB
PSRAM	Undetermined	2 MB	Not available
무선통신	Wi-Fi 4, Bluetooth Classic	Wi-Fi 4, Bluetooth 5 LE	

되도록 작은 개발보드로 만들고 싶다!



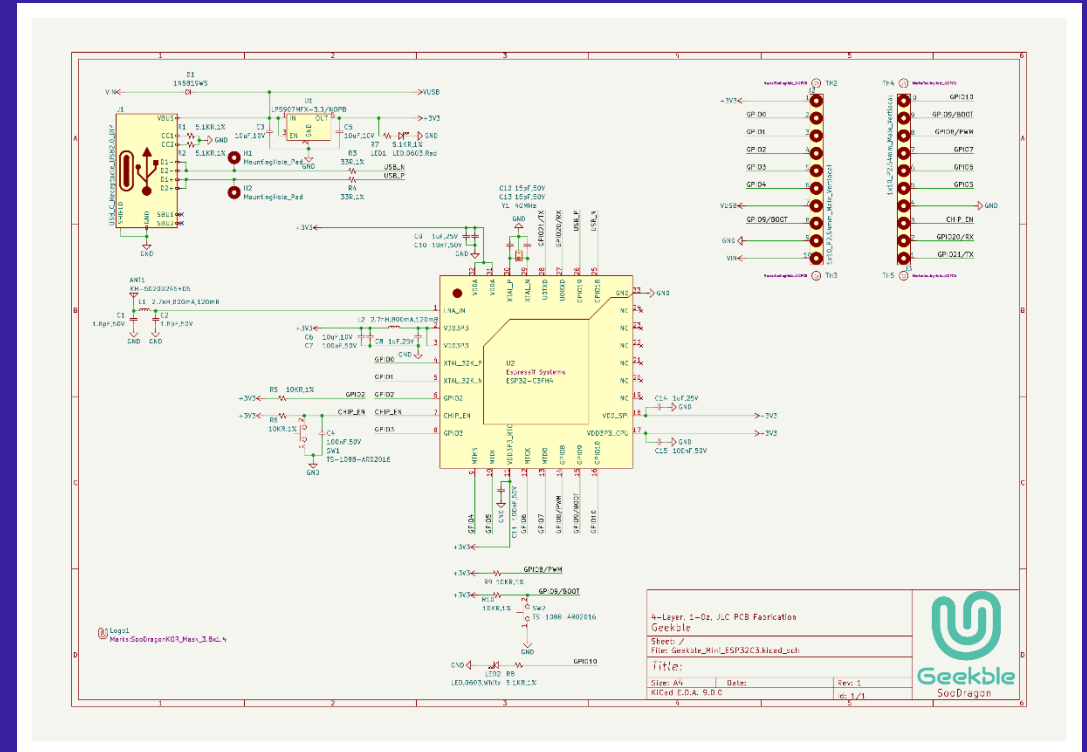
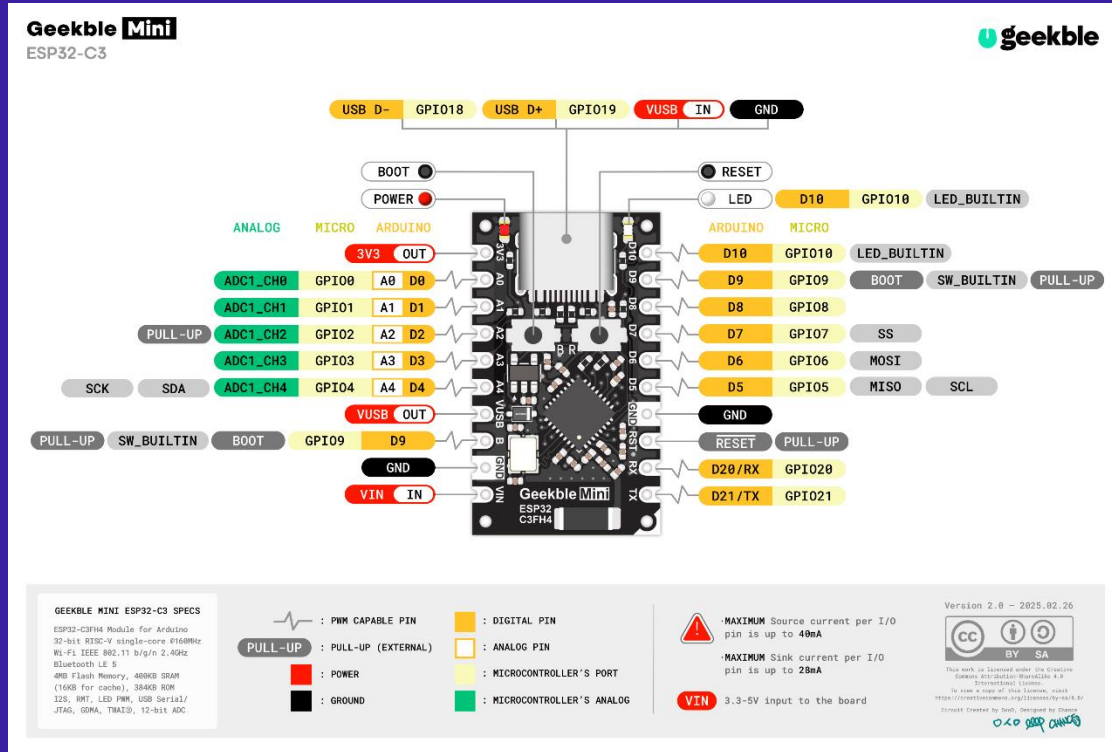
ESP32-C3 Super Mini 결함 이슈

- 안테나 위치 설계 오류로 인한 Wi-Fi 성능 저하
- 외장 안테나 DIY로 해결 가능하지만 번거로움



<https://youtu.be/UHTdhCrSA3g>

최종 선택: Geekble Mini



- 오픈소스: KiCad 설계 파일과 Gerber, 부품 리스트가 공개되어 있음

nanoff 설치

- **.NET nanoFramework firmware flasher**
- 펌웨어 이미지 설치 및 사용자 애플리케이션 배포 도구
- MCU 사양 확인 및 적정 펌웨어 자동 다운로드 기능 제공
- 설치 방법

```
dotnet tool install -g nanoff
```

nanoff로 펌웨어 설치하기

- 자동 설치

```
nanoff --platform esp32 --serialport COM7 --update
```

- 특정 타겟 선택

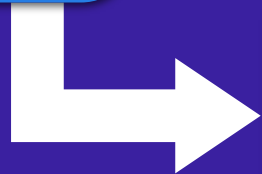
```
nanoff --platform esp32 --serialport COM7 --update --target XIA0_ESP32C3
```

에어컨 리모컨 만들기

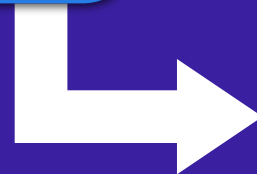


작업 개요

기존 리모컨의
적외선 신호 수집



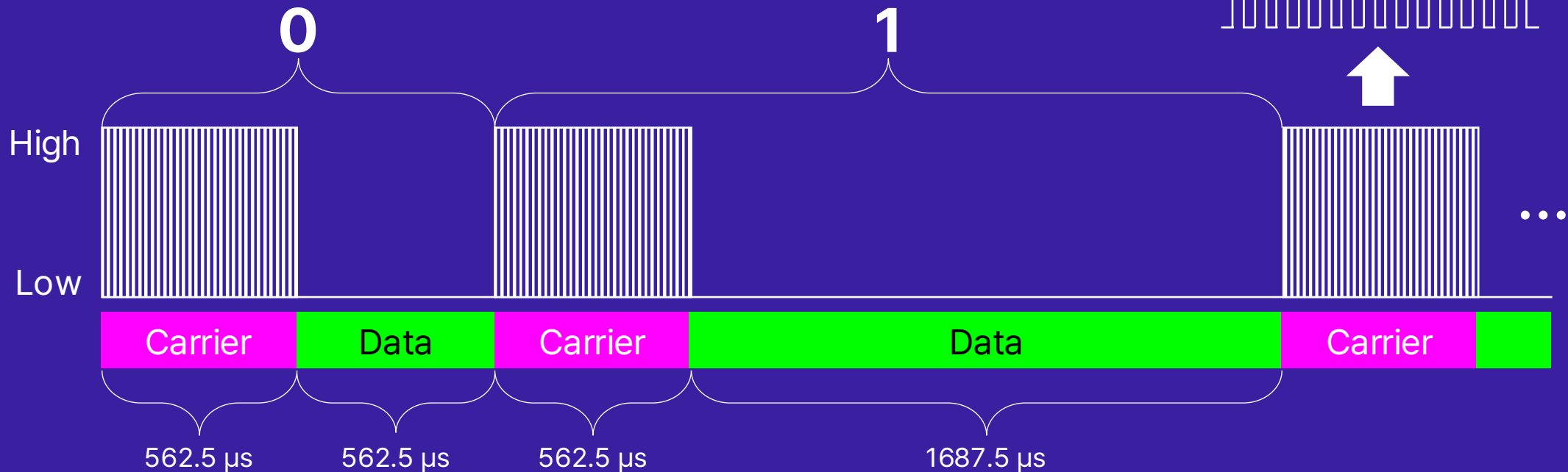
적외선 신호 분석 및
리모컨 커맨드 정리



리모컨 커맨드에 따른
적외선 신호 재생

리모컨 신호의 기본 구조

- 적외선 리모컨의 **NEC IR 프로토콜**은 마이크로초 단위의 신호처리가 필요함



RMT (Remote Control Transceiver)

- 적외선 리모컨 신호의 송수신을 위해 설계된 ESP32 내장 기능
- 나노초 단위의 정밀 펄스 제어 가능
- 적외선 리모컨 뿐만 아니라 다양하게 응용 가능
 - 예: WS2812B/WS2815 LED 색상 제어, DHT 온습도 센서 데이터 수집
- NuGet 패키지: **`nanoFramework.Hardware.Esp32.Rmt`**

리모컨 신호 수신 예제

```
using nanoFramework.Hardware.Esp32.Rmt;  
using System.Diagnostics;
```

```
...
```

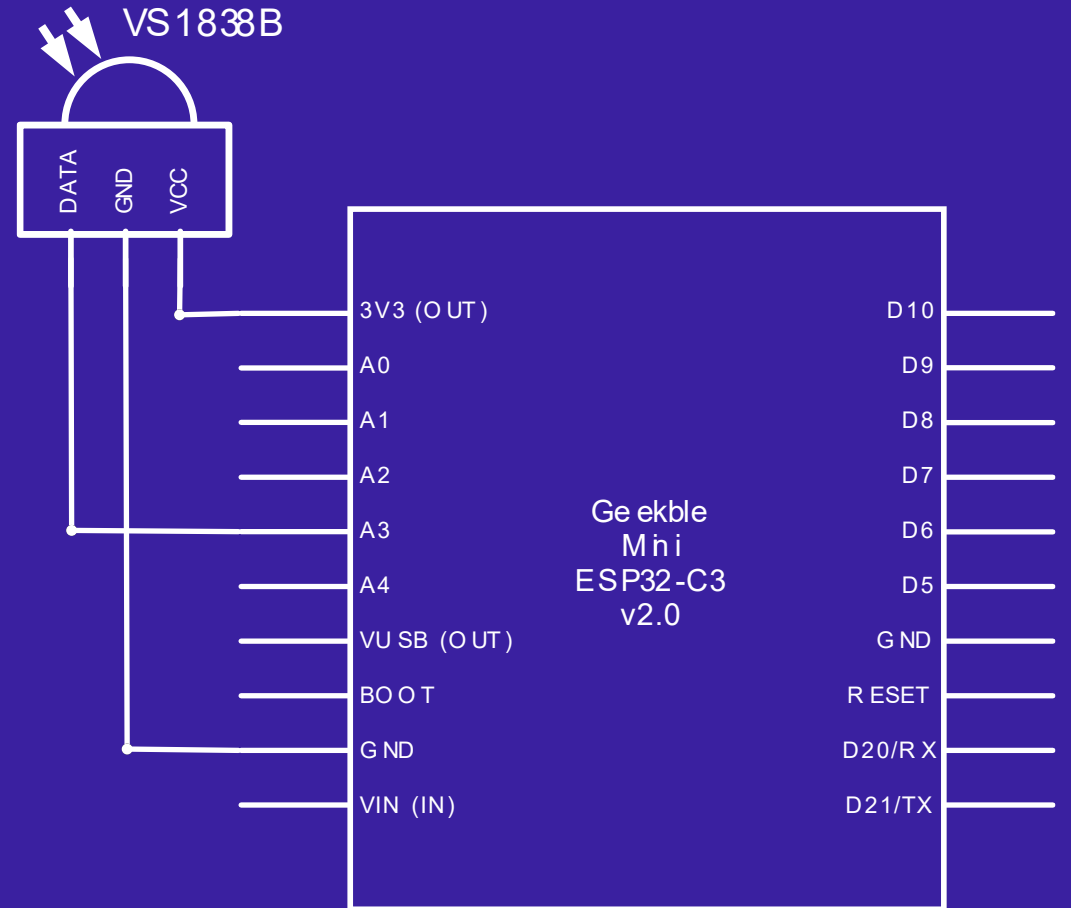
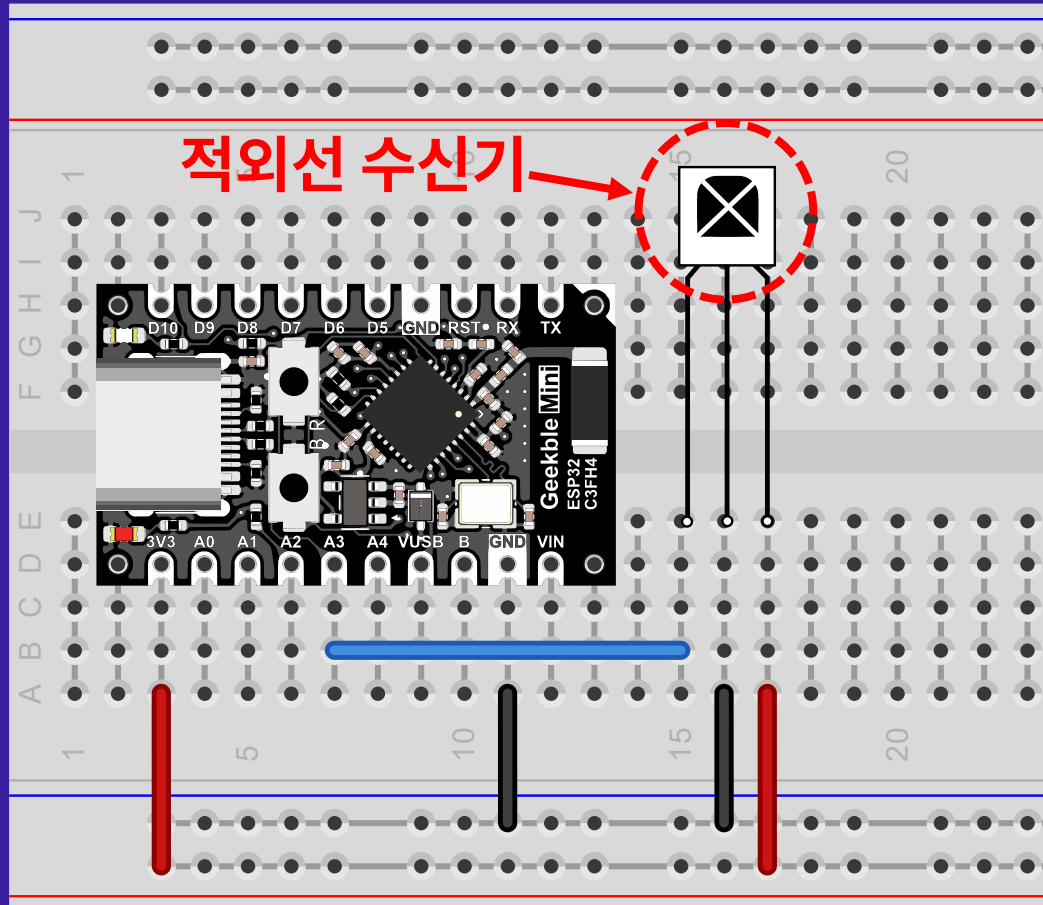
```
ReceiverChannel rx = new(new ReceiverChannelSettings(pinNumber: 3));  
rx.Start(true);
```

```
while (true)  
{  
    var commands = rx.GetAllItems();  
    if (commands != null)  
        foreach (var cmd in commands)  
            Debug.WriteLine($"{cmd.Duration0}, {cmd.Duration1}");  
}
```

MCU의 GPIO 핀 번호 설정



리모컨 신호 수신을 위한 하드웨어 구성



예외 발생

The screenshot displays the Visual Studio IDE with a C# project named 'RemoteAC'. The code in 'Program.cs' defines a 'Program' class with a 'Main' method. Line 10, where 'ReceiverChannel' is instantiated, is highlighted. A red error icon is present next to this line.

An exception dialog box is open, stating: '예외가 처리되지 않음' (Exception not handled). The message reads: '처리되지 않은 'System.Exception' 형식의 예외가 nanoFramework.Hardware.Esp32.Rmt.dll에서 발생했습니다.' (An unhandled exception of type 'System.Exception' occurred in nanoFramework.Hardware.Esp32.Rmt.dll).

The '출력' (Output) window at the bottom left shows the following log entries:

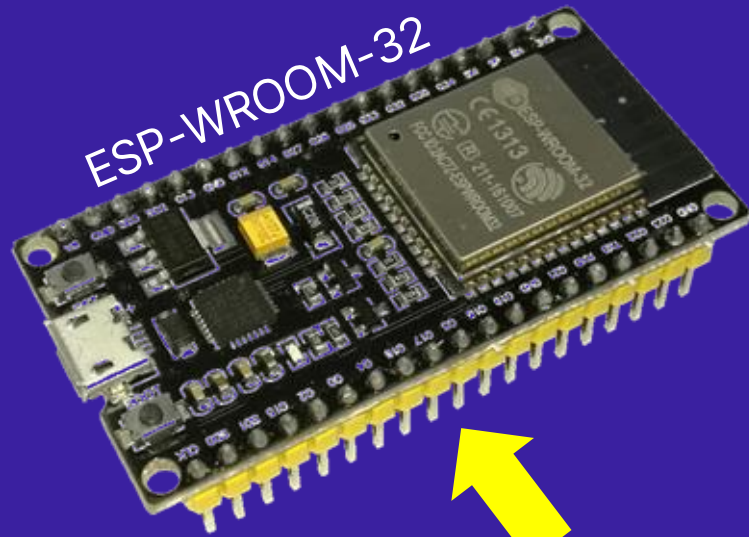
```
03:42.117 '이름 없음' (2) 스레드가 종료되었습니다(코드: 0 (0x0)).
03:42.867 +++ Exception System.Exception - CLR_EXCEPTION_NOT_REGISTERED (1) +++
03:42.867 +++ Message:
03:42.867 +++ nanoFramework.Hardware.Esp32.Rmt.ReceiverChannel::NativeRxInit [IP: 0000] +++
03:42.867 +++ nanoFramework.Hardware.Esp32.Rmt.ReceiverChannel::.ctor [IP: 001a] +++
03:42.867 +++ DumpRemoteControl.Program::Main [IP: 0008] +++
03:43.379 예외 발생: 'System.Exception' (nanoFramework.Hardware.Esp32.Rmt.dll)
03:44.128 처리되지 않은 'System.Exception' 형식의 예외가 nanoFramework.Hardware.Esp32.Rmt.dll에서 발생했습니다.
```

The '호출 스택' (Call Stack) window at the bottom right shows the following stack trace:

```
검색(Ctrl+E)
이름
[외부 코드]
DumpRemoteControl.ESP32.exe!DumpRemoteControl.Program.Main() 줄 10 + 0x9 바이트
C#
```

The status bar at the bottom indicates '준비' (Ready).

문제점 분석



저것은 잘 되는데, 이것은 안된다?

ESP32

ESP32-C3

문제점 해결

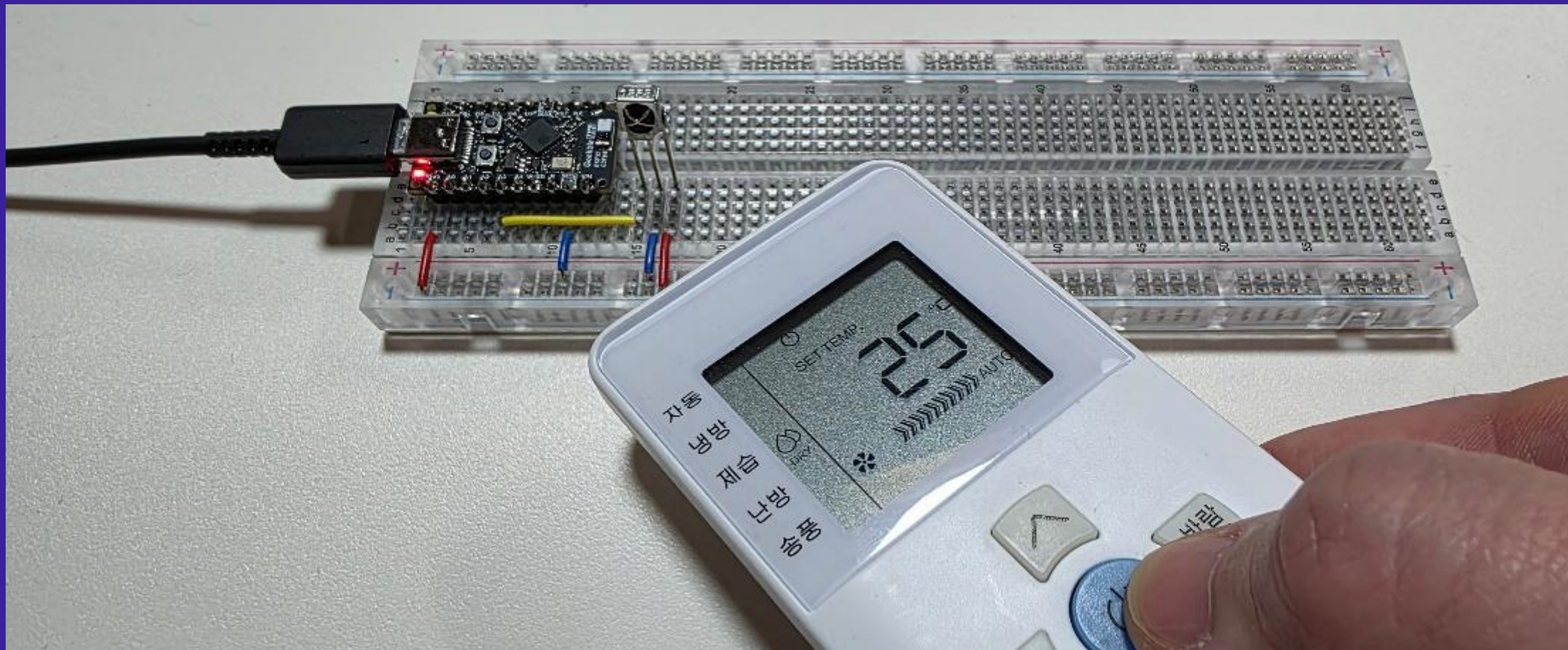
```
ReceiverChannel rx = new(new ReceiverChannelSettings(channel: 2, pinNumber: 3));  
rx.Start(true);
```

```
while (true)  
{  
    ...  
}
```

채널 번호 설정 추가

- ESP32-C3의 **0~1**번 채널은 송신용, **2~3**번 채널은 수신용으로 설계됨
- ESP32-S3은 **0~3**번, **4~7**번 채널이 각각 송신, 수신용으로 설계됨
- ESP32는 **모든 채널이 송/수신 겸용**이므로 채널 설정 필요 없음

리모컨 신호 훔치기

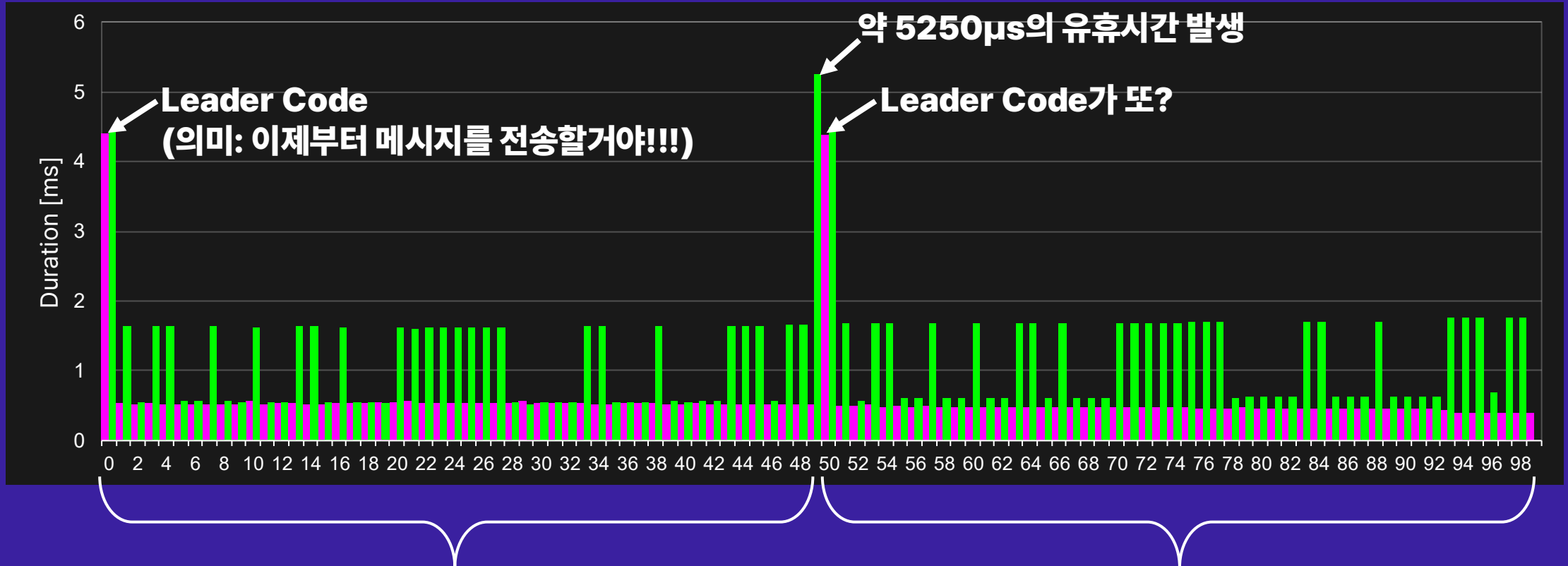
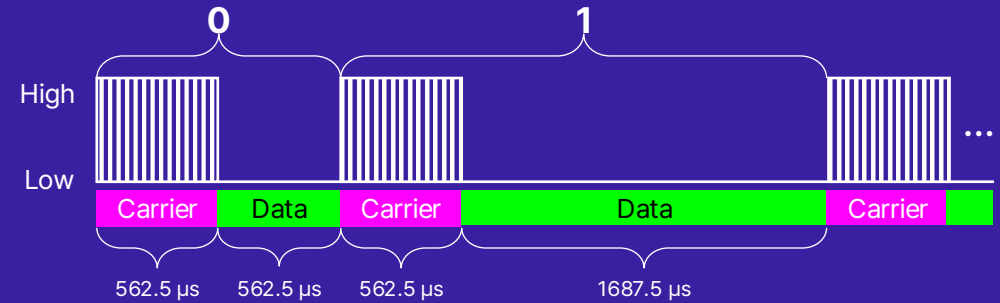


또 문제가 발생...

- 같은 버튼을 반복해서 눌렀는데, 신호가 오다가 안 오다가 오락가락 함
- 게다가 정상 수신할 때마다 신호 개수가 달라서 일관성이 없음
- 리모컨에서 한번에 많은 신호를 전송할 경우 링 버퍼 오버플로우 발생
- 이런 현상이 발생한다면 버퍼 크기(기본값 100)를 넉넉하게 설정

```
ReceiverChannel rx = new(new ReceiverChannelSettings(channel:2, pinNumber: 3)
{
    BufferSize = 300 //High, Low 한 쌍으로 150개까지 받도록 설정
});
rx.Start(true);
...
```

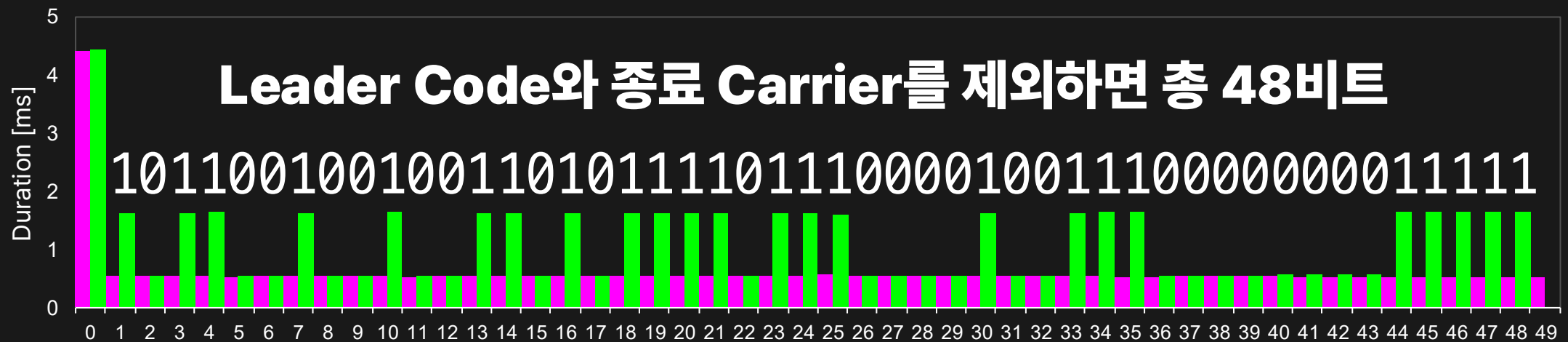
리모컨 신호 수신 결과



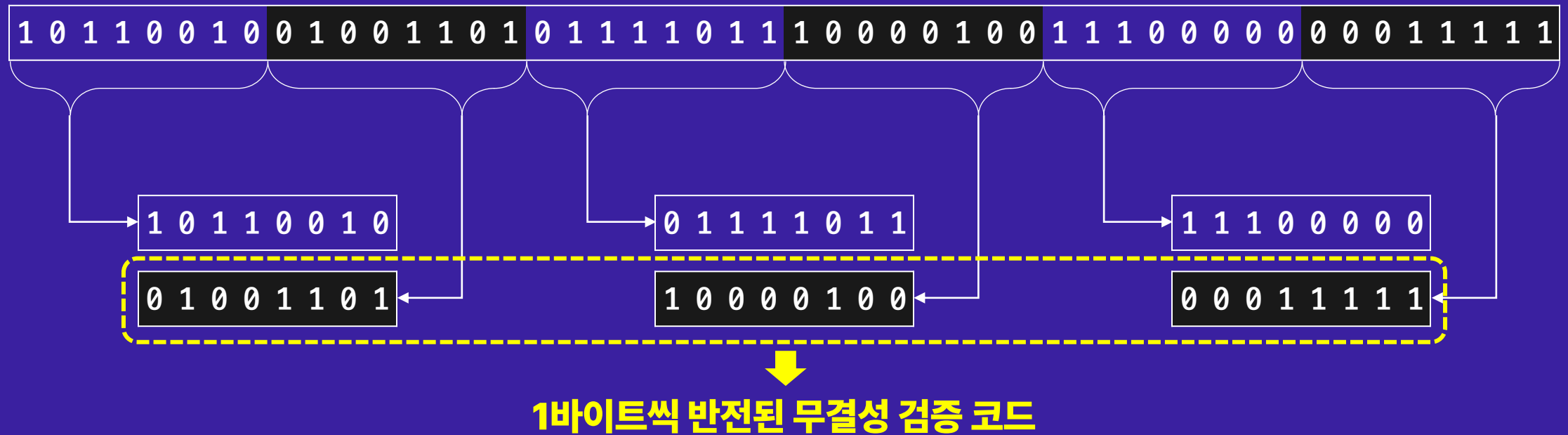
리모컨 버튼을 한 번 누를 때, 동일한 메시지를 두 번 송신함

유휴시간 설정으로 반복 메시지 분리 가능

```
ReceiverChannel rx = new(new ReceiverChannelSettings(channel:2, pinNumber: 3)
{
    IdleThreshold = 5000,    //메시지 구분을 위한 유휴시간 설정
    BufferSize = 110        //하나의 메시지가 50개의 신호로 구성되므로 축소 가능
});
```



NEC IR 프로토콜의 무결성 검증 코드



- 즉, 리모컨의 메시지는 48비트의 절반인 24비트로 구성됨

무결성 검증 및 24비트만 추출하기

```
void PrintMessage(RmtCommand[] commands)
{
    bool ToLevel(int i, int j, int offset = 0)
        => commands[i * 16 + j + 1 + offset].Duration1 > 1125;

    if (commands != null && commands.Length >= 50)
    {
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 8; j++)
                if (ToLevel(i, j) == ToLevel(i, j, 8))
                    return;

        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 8; j++)
                Debug.Write($"{(ToLevel(i, j) ? 1 : 0)}");

        Debug.WriteLine();
    }
}
```

```
ReceiverChannel rx = new(
    new ReceiverChannelSettings(channel:2, pinNumber: 3)
    { BufferSize = 300 });
rx.Start(true);

while (true)
{
    PrintMessage(rx.GetAllItems());
}
```

출력

출력 보기 선택(S): 디버그

46: 59.338 ' <이름 없음>' (2) 스레드가 종료되었습니다(코드: 0 (0x0)).

47: 06.421 101100100111101111100000

47: 07.919 101100100001111111000100

47: 10.920 101100100111101111100000

47: 12.917 101100100001111111000100

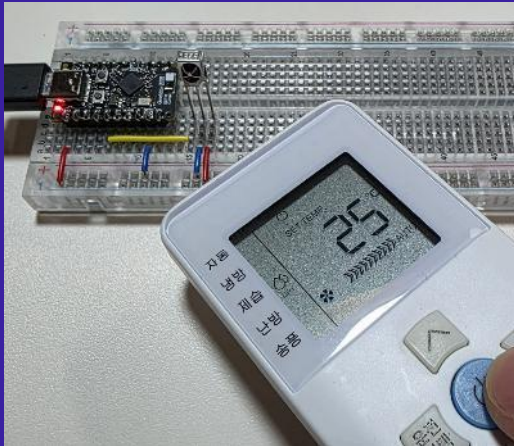
47: 15.708 101100100111101111100000

47: 20.499 101100100001111111000100

오류 목록 출력 로컬 조사식 1

리모컨 메시지 수집

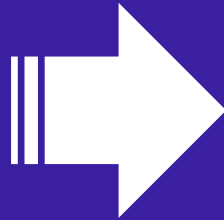
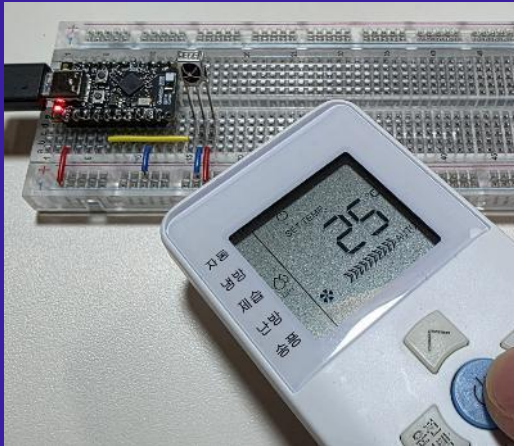
24비트 리모컨 메시지:



1	0	1	1	0	0	1	0	0	1	1	1	1	0	1	1	1	1	1	0	0	0	0	0	0
1	0	1	1	0	0	1	0	1	0	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0
1	0	1	1	0	0	1	0	0	0	0	1	1	1	1	1	1	1	0	0	0	1	0	0	0
1	0	1	1	0	0	1	0	1	0	1	1	1	1	1	1	1	1	0	0	1	1	0	0	0
1	0	1	1	0	0	1	0	1	0	1	1	1	1	1	1	1	1	1	0	0	1	0	0	0
1	0	1	1	0	0	1	0	0	1	0	1	1	1	1	1	1	0	0	1	0	0	0	0	0
1	0	1	1	0	0	1	0	1	0	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0
1	0	1	1	0	0	1	0	1	0	0	1	1	1	1	1	1	1	0	0	0	0	0	0	0
1	0	1	1	0	0	1	0	1	0	1	1	1	1	1	1	1	0	0	1	0	0	0	0	0
1	0	1	1	0	0	1	0	1	0	1	1	1	1	1	1	1	0	1	1	0	0	0	0	0
1	0	1	1	0	0	1	0	1	0	1	1	1	1	1	1	1	0	1	0	0	0	0	0	0

리모컨 메시지 분석

24비트 리모컨 메시지: 1 0 1 1 0 0 1 0 0 1 1 1 1 0 1 1 1 1 0 0 0 0 0



풍량	Bits		
미풍	1	0	0
약풍	0	1	0
강풍	0	0	1
자동풍	1	0	1
Zero	0	0	0
Off	0	1	1

모드가 자동이거나
제습일 때 사용함

전원	Bit
On	1
Off	0

온도	Bits			
17°C	0	0	0	0
18°C	0	0	0	1
19°C	0	0	1	1
20°C	0	0	1	0
21°C	0	1	1	0
22°C	0	1	1	1
23°C	0	1	0	1
24°C	0	1	0	0
25°C	1	1	0	0
26°C	1	1	0	1
27°C	1	0	0	1
28°C	1	0	0	0
29°C	1	0	1	0
30°C	1	0	1	1
Off	1	1	1	0

모드	Bits	
자동	1	0
냉방	0	0
제습	0	1
난방	1	1
송풍	0	1
Off	0	0

- 모든 설정 값들을 하나의 메시지로 한번에 전송함

리모컨 메시지 데이터 정의

17°C	0	0	0	0	0
18°C	0	0	0	1	1
19°C	0	0	1	1	3
20°C	0	0	1	0	2
21°C	0	1	1	0	6
22°C	0	1	1	1	7
23°C	0	1	0	1	5
24°C	0	1	0	0	4
25°C	1	1	0	0	12
26°C	1	1	0	1	13
27°C	1	0	0	1	9
28°C	1	0	0	0	8
29°C	1	0	1	0	10
30°C	1	0	1	1	11

```
byte[] tempCodes = new byte[]  
    { 0, 1, 3, 2, 6, 7, 5, 4, 12, 13, 9, 8, 10, 11 };
```

```
byte EncodeTemperature(int temperature)  
    => tempCodes[Math.Clamp(temperature, 17, 30) - 17];
```

```
enum OpMode : byte
```

```
{
```

```
    Auto = 0b0_10,
```

```
    Cool = 0b0_00,
```

```
    Dry = 0b1_01,
```

```
    Heat = 0b0_11,
```

```
    Fan = 0b0_01
```

```
}
```

자동	1	0
냉방	0	0
제습	0	1
난방	1	1
송풍	0	1

```
enum FanSpeed : byte
```

```
{
```

```
    Low = 0b100,
```

```
    Mid = 0b010,
```

```
    High = 0b001,
```

```
    Auto = 0b101
```

```
}
```

미풍	1	0	0
약풍	0	1	0
강풍	0	0	1
자동풍	1	0	1

에어컨 리모컨 클래스 정의

```
public class AirconRemoteController
{
    public AirconRemoteController(int channel, int pinNumber)
    {
        tx = new TransmitterChannel(new TransmitChannelSettings(channel, pinNumber)
        {
            BufferSize = 300 //버퍼는 넉넉하게 300으로 설정
        });
    }

    private readonly TransmitterChannel tx; //RMT 라이브러리의 적외선 신호 송출 클래스

    private bool isRun = false;
    private OpMode opMode = OpMode.Auto;
    private int temperature = 25;
    private FanSpeed fanSpeed = FanSpeed.Auto;

    public bool IsRun { get => isRun; set { isRun = value; Send(); } }
    public OpMode OpMode { get => opMode; set { opMode = value; Send(); } }
    public int Temperature { get => temperature; set { temperature = value; Send(); } }
    public FanSpeed FanSpeed { get => fanSpeed; set { fanSpeed = value; Send(); } }

    private void Send() => Send(isRun, opMode, temperature, fanSpeed);
    ...
}
```

에어컨 제어 속성 정의

리모컨 메시지 생성

```
private static byte[] BuildMessage(FanSpeed fanSpeed, bool run, int temperature, OpMode opMode)
{
```

```
    byte fanSpeedCode = (byte)fanSpeed;
```

```
    byte temperatureCode = EncodeTemperature(temperature);
```

```
    byte opModeCode = (byte)((byte)opMode & 0b11);
```

```
    if (!run)
```

```
    { //에어컨을 끌 때는 풍량, 온도, 모드를 특정 값으로 강제함
```

```
        fanSpeedCode = 0b011;
```

```
        temperatureCode = 0b1110;
```

```
        opModeCode = 0b00;
```

```
    }
```

```
    else
```

```
        switch (opMode)
```

```
        {
```

```
            case OpMode.Auto: //모드가 자동이거나 제습일 때는 풍량을 특정 값으로 강제함
```

```
            case OpMode.Dry: fanSpeedCode = 0b000; break;
```

```
            case OpMode.Fan: temperatureCode = 0b1110; break; //모드가 송풍이면 온도값 Off로 설정
```

```
        }
```

```
    byte[] msg = new byte[] { 0b10110010, 0b00011011, 0b00000000 };
```

```
    msg[1] |= (byte)(fanSpeedCode << 5);
```

```
    msg[1] |= (byte)((run ? 1 : 0) << 2);
```

```
    msg[2] |= (byte)(temperatureCode << 4);
```

```
    msg[2] |= (byte)(opModeCode << 2);
```

```
    return msg;
```

```
}
```

풍량	Bits		
미풍	1	0	0
약풍	0	1	0
강풍	0	0	1
자동풍	1	0	1
Zero	0	0	0
Off	0	1	1

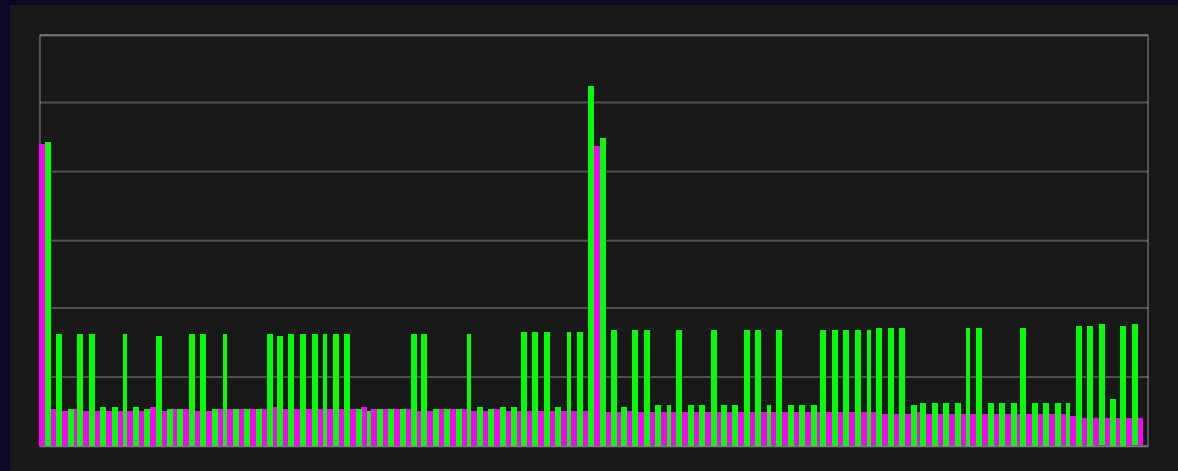
온도	Bits			
17°C	0	0	0	0
18°C	0	0	0	1
19°C	0	0	1	1
20°C	0	0	1	0
21°C	0	1	1	0
22°C	0	1	1	1
23°C	0	1	0	1
24°C	0	1	0	0
25°C	1	1	0	0
26°C	1	1	0	1
27°C	1	0	0	1
28°C	1	0	0	0
29°C	1	0	1	0
30°C	1	0	1	1
Off	1	1	1	0

모드	Bits	
자동	1	0
냉방	0	0
제습	0	1
난방	1	1
송풍	0	1
Off	0	0

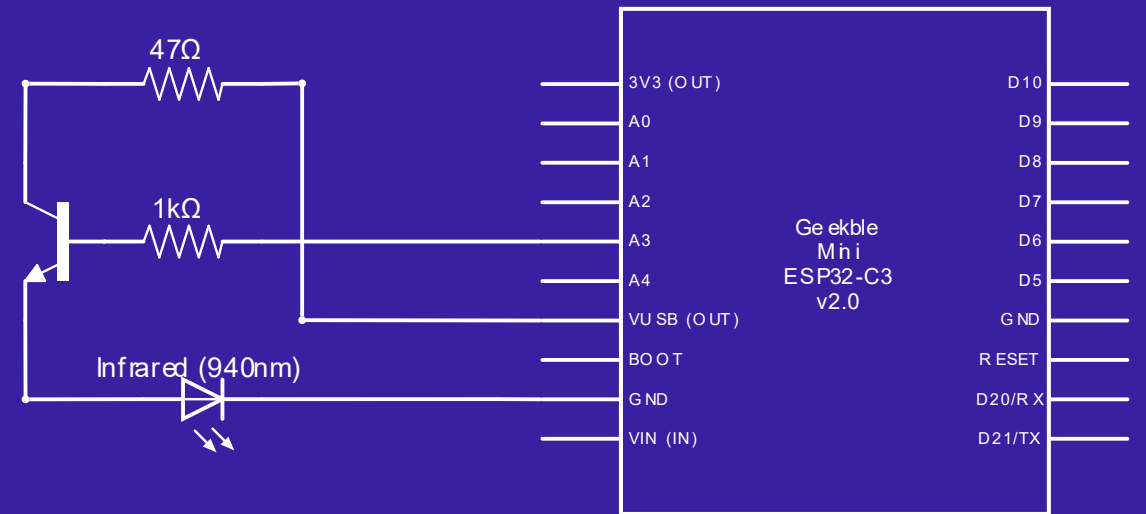
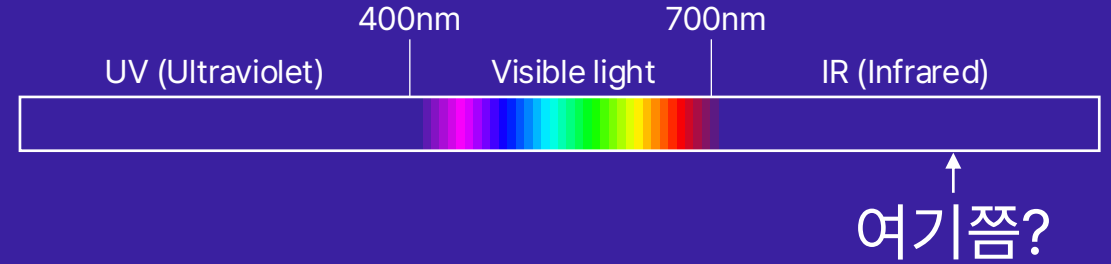
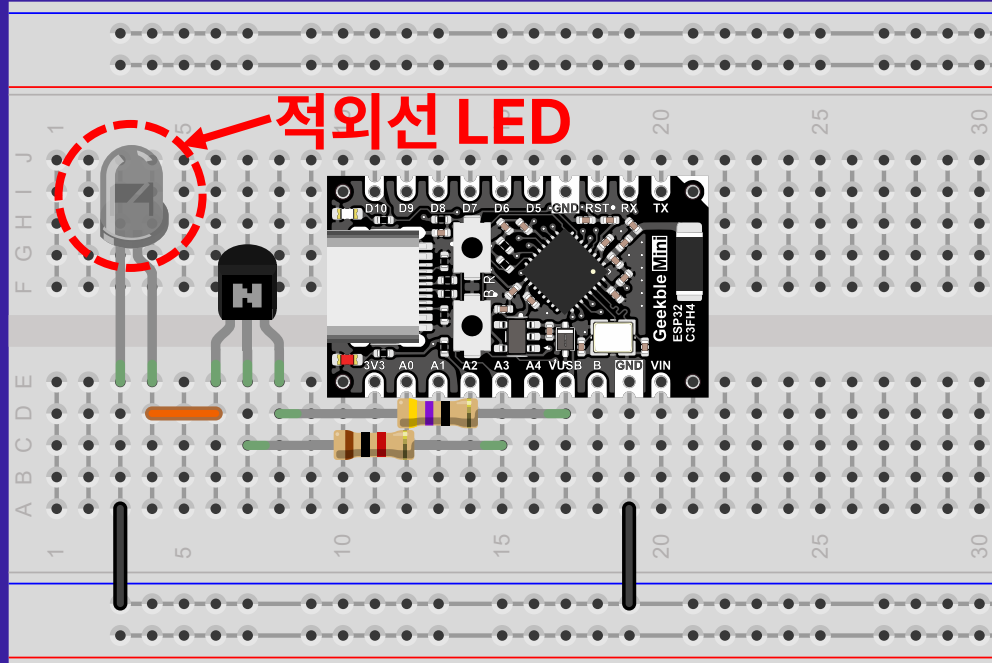
적외선 신호 송출

```
private void Send(bool run, OpMode opMode, int temperature, FanSpeed fanSpeed)
{
    var msg = BuildMessage(fanSpeed, run, temperature, opMode); //24비트 에어컨 리모컨 메시지 생성

    void AddCommands(byte data) //byte 값을 이용한 8비트 신호 생성 메서드
    {
        for (int i = 7; i >= 0; i--)
            tx.AddCommand(new RmtCommand(560, true, (ushort)(((data >> i) & 1) == 1 ? 1690 : 560), false));
    }
    lock (tx)
    {
        tx.ClearCommands(); //버퍼의 RMT 커맨드 비우기
        for (int i = 0; i < 2; i++)
        {
            tx.AddCommand(new RmtCommand(4500, true, 4500, false));
            foreach (byte data in msg)
            {
                AddCommands(data);
                AddCommands((byte)~data); //무결성 검증 코드 생성
            }
            tx.AddCommand(new RmtCommand(560, true, 5200, false));
        }
        tx.Send(true); //버퍼에 추가된 RMT 커맨드 일괄 송출
    }
}
```

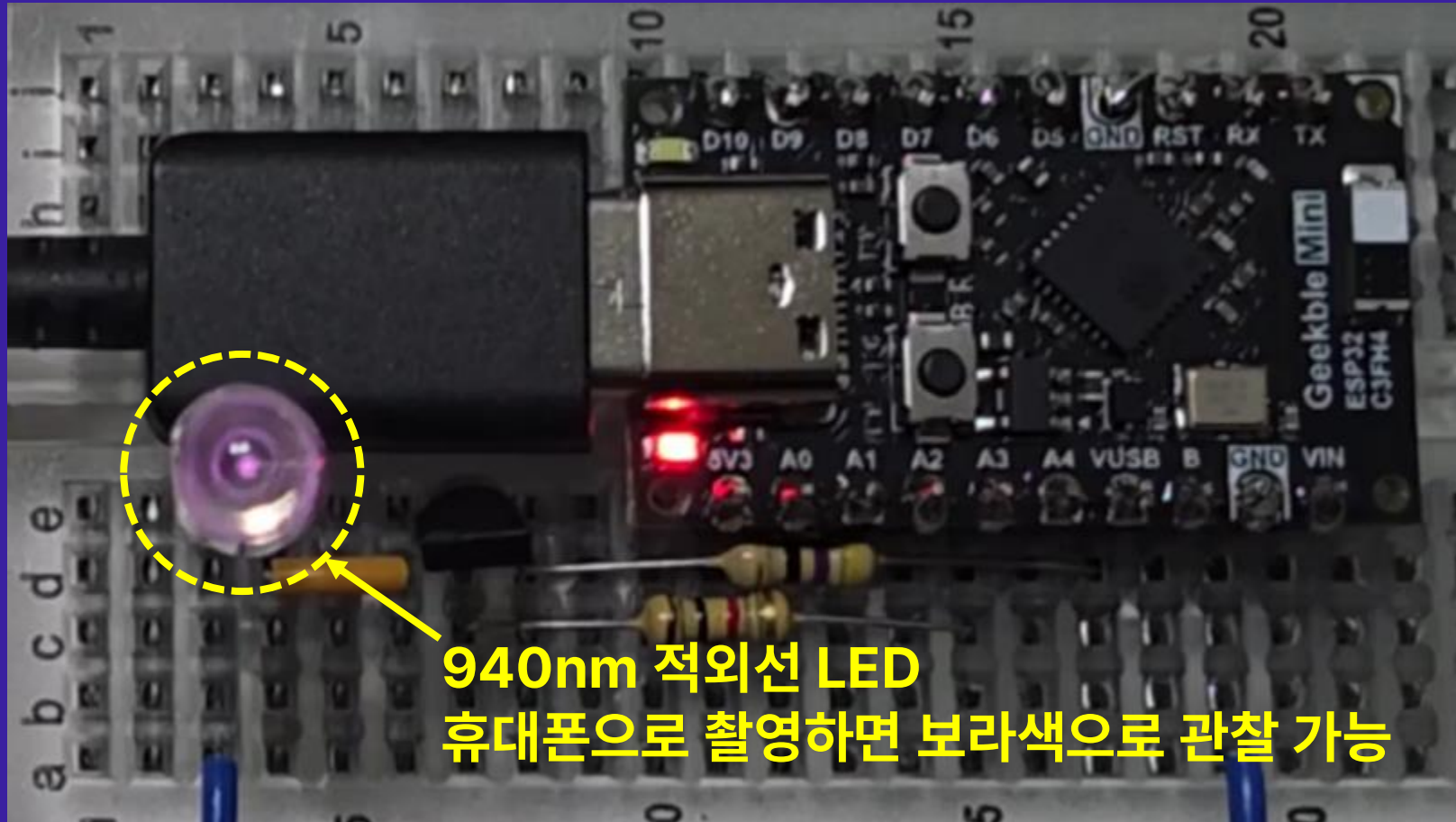


적외선 신호 송출 회로



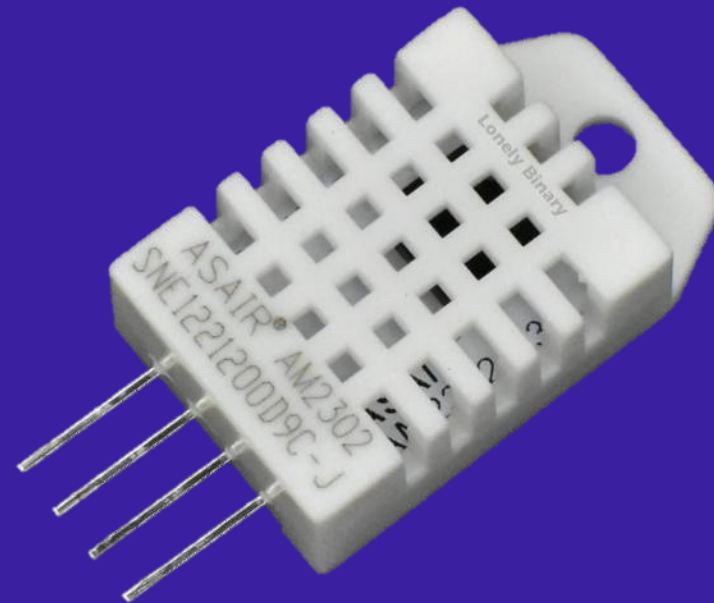
- 전원으로 USB 전압을 사용하고, 트랜지스터와 MCU의 GPIO핀으로 제어
- 940nm 파장의 적외선 LED로 신호 송출

적외선 신호 송출 테스트



부가 기능: 주변 온도 및 습도 측정하기

- 주변 온도와 습도에 따른 에어컨 제어
- DHT22 센서 모듈
 - 온도 측정 범위: $-40 \sim 80\text{ }^{\circ}\text{C}$
 - 습도 측정 범위: $0 \sim 99.9\%$
 - 온도 정확도: $\pm 0.5\text{ }^{\circ}\text{C}$
 - 습도 정확도: $\pm 2\%$
 - 인터페이스 방식: 1-Wire 통신



DHT22 관련 클래스 및 예제 코드

The screenshot shows the nanoFramework API Reference website. The left sidebar contains a tree view of the API, with 'Dht22' selected under 'IoT.Device.DHTxx.Esp32'. The main content area displays the 'Class Dht22' documentation. It includes the namespace 'IoT.Device.DHTxx.Esp32', the assembly 'IoT.Device.Dhtxx.Esp32.dll', and a description 'Temperature and Humidity Sensor DHT22'. The class definition is shown as 'public class Dht22 : DhtBase'. Under 'Inheritance', it shows 'object ← DhtBase ← Dht22'. Under 'Inherited Members', it lists various methods and properties from the base class. The 'Constructors' section shows 'Dht22(int, int, PinNumberingScheme, GpioController?, bool)' and a description 'Create a DHT22 sensor'. The constructor signature is also shown in a code block: 'public Dht22(int pinEcho, int pinTrigger, PinNumberingScheme pinNumberingScheme = ...'.

```
using Iot.Device.DHTxx.Esp32;
using System.Diagnostics;
using System.Threading;

...

using Dht22 dht = new(pinEcho: 0, pinTrigger: 1);

while (true)
{
    var temp = dht.Temperature;
    var humi = dht.Humidity;
    if (dht.IsLastReadSuccessful)
    {
        Debug.WriteLine($"T: {temp.Value} °C, H: {humi.Value} %");
    }
    else
    {
        Debug.WriteLine("Error reading DHT sensor");
    }
    Thread.Sleep(5000);
}
```

예외 발생

적외선 신호 수신 예제와 똑같은 예외가 발생했다?

예외가 처리되지 않음
처리되지 않은 'System.Exception' 형식의 예외가 nanoFramework.Hardware.Esp32.Rmt.dll에서 발생했습니다.

출력
출력 보기 선택(S): 디버그
31:58.882 '이름 없음' (2) 스레드가 종료되었습니다(코드: 0 (0x0)).
31:59.392 + Exception System.Exception - CLR_E_DRIVER_NOT_REGISTERED (1) +
31:59.392 + Message:
31:59.392 + nanoFramework.Hardware.Esp32.Rmt.ReceiverChannel::NativeRxInit [IP: 0000] +
31:59.392 + nanoFramework.Hardware.Esp32.Rmt.ReceiverChannel::ctor [IP: 001a] +
31:59.392 + IoT.Device.DHTxx.Esp32.DhtBase::ctor [IP: 0069] +
31:59.392 + IoT.Device.DHTxx.Esp32.Dht22::ctor [IP: 000d] +
31:59.392 + RemoteAC.Program::Main [IP: 000a] +
31:59.890 예외 발생: 'System.Exception' (nanoFramework.Hardware.Esp32.Rmt.dll)
32:00.435 처리되지 않은 'System.Exception' 형식의 예외가 nanoFramework.Hardware.Esp32.Rmt.dll에서 발생했습니다.

호출 스택
예외 설정
이름
[외부 코드]
RemoteAC.ESP32.exe!RemoteAC.Program.Main() 줄 11 + 0x9 바이트
C#
호출 스택 예외 설정 직접 실행 창

DHT 관련 라이브러리 소스 분석

```
100    /// <param name="gpioController"><see cref="GpioController"/> related with operations on pins</param>
101    /// <param name="shouldDispose"><see langword="true"/> to dispose the <see cref="GpioController"/></param>
102    public DhtBase(int pinEcho, int pinTrigger, PinNumberingScheme pinNumberingScheme = PinNumberingScheme.Logical, GpioController? gpioController = null, bool shouldDispose = true)
103    {
104        _protocol = CommunicationProtocol.OneWire;
105        _shouldDispose = shouldDispose || gpioController is null;
106        _controller = gpioController ?? new GpioController(pinNumberingScheme);
107        _pin = pinTrigger;
108
109        var rxChannelSettings = new ReceiverChannelSettings(pinNumber: pinEcho)
110        {
111            // 1us clock ( 80Mhz / 80 ) = 1Mhz
112            ClockDivider = 80,
113
114            // no filter
115            EnableFilter = false,
116            FilterThreshold = 5,
117
118            // max time 1us clock
119            IdleThreshold = ushort.MaxValue,
120
121            // timeout after 1 second
122            ReceiveTimeout = TimeSpan.FromSeconds(1)
123        };
124
125        _rxChannel = new ReceiverChannel(rxChannelSettings);
126        _controller.OpenPin(_pin, PinMode.Output);
127
128        // delay 1s to make sure DHT stable
129        Thread.Sleep(1000);
130    }
131
```

RMT 채널 설정을 할 수 없음...
ESP32-S3, ESP32-C3는 어쩌라고...

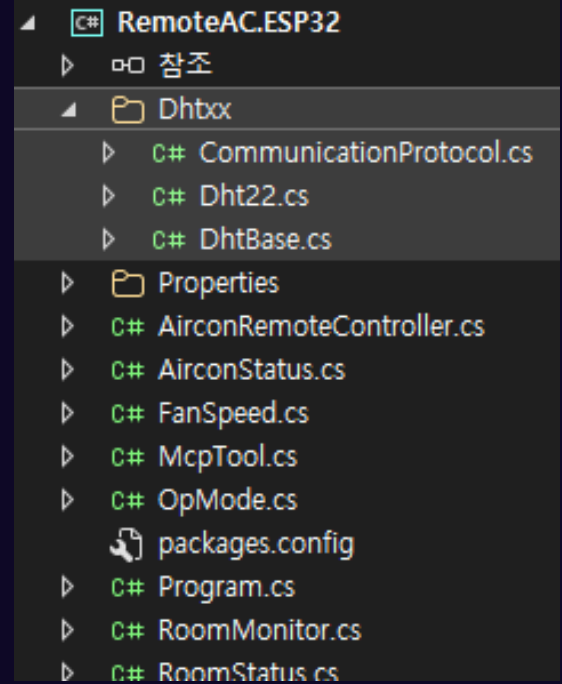
필요한 소스만 갖고 와서 고쳐 쓰자!

```
public Dht22(int pinEcho, int pinTrigger,
    PinNumberingScheme pinNumberingScheme = PinNumberingScheme.Logical,
    GpioController? gpioController = null, bool shouldDispose = true, int rmtChannel = -1)
    : base(pinEcho, pinTrigger, pinNumberingScheme, gpioController, shouldDispose, rmtChannel)
{ }
```

RMT 채널 설정 매개변수 추가

```
...
public DhtBase(int pinEcho, int pinTrigger,
    PinNumberingScheme pinNumberingScheme = PinNumberingScheme.Logical,
    GpioController? gpioController = null, bool shouldDispose = true, int rmtChannel = -1)
{
    _protocol = CommunicationProtocol.OneWire;
    _shouldDispose = shouldDispose || gpioController is null;
    _controller = gpioController ?? new GpioController(pinNumberingScheme);
    _pin = pinTrigger;

    var rxChannelSettings = new ReceiverChannelSettings(channel: rmtChannel, pinNumber: pinEcho)
    {
        // 1us clock ( 80Mhz / 80 ) = 1Mhz
        ClockDivider = 80,
    }
    ...
```



기왕 하는 거 오픈소스에 기여해보자!

Commit 23db675

Exposed RMT channel configuration for libraries using ESP32 RMT (#1473)

12 files changed +53 -25 lines changed

RMT 라이브러리를 사용하는 모든 클래스 수정

```
@@ -19,8 +19,9 @@ public class Dht11 : DhtBase
19 19    /// <param name="pinNumberingScheme">The GPIO pin numbering scheme</param>
20 20    /// <param name="gpioController"><see cref="GpioController"/> related with operations on pins</param>
21 21    /// <param name="shouldDispose"><see langword="true"/> to dispose the <see cref="GpioController"/></param>
22 -    public Dht11(int pinEcho, int pinTrigger, PinNumberingScheme pinNumberingScheme = PinNumberingScheme.Logical, GpioController? gpioController = null, bool
      shouldDispose = true)
23 -    : base(pinEcho, pinTrigger, pinNumberingScheme, gpioController, shouldDispose)
22 +    /// <param name="rmtChannel">The RMT channel number to use. Valid value range is 0 to 7 (inclusive), or -1 to auto-select a free channel.</param>
23 +    public Dht11(int pinEcho, int pinTrigger, PinNumberingScheme pinNumberingScheme = PinNumberingScheme.Logical, GpioController? gpioController = null, bool
      shouldDispose = true, int rmtChannel = -1)
24 +    : base(pinEcho, pinTrigger, pinNumberingScheme, gpioController, shouldDispose, rmtChannel)
24 25    {
25 26    }
26 27
```

Pull request, Merge, NuGet 배포 완료

The screenshot shows a GitHub pull request interface. At the top, the browser address bar displays the URL: `https://github.com/nanoframework/nanoFramework.IoT.Device/pull/1473`. The repository path is `nanoframework / nanoFramework.IoT.Device`. The pull request title is "Exposed RMT channel configuration for libraries using ESP32 RMT #1473". It is marked as "Merged" and shows it was merged 14 commits into `nanoframework:main` from `Vagabond-K:develop` 12 hours ago. The interface includes tabs for Conversation (27), Commits (14), Checks (2), and Files changed (12). A detailed description of the pull request is provided, explaining the need for compatibility with ESP32-S and ESP32-C series. The right sidebar shows the review process, with Copilot and josesimoes as reviewers, and no assignees or labels assigned.

Exposed RMT channel configuration for libraries using ESP32 RMT #1473

Merged josesimoes merged 14 commits into `nanoframework:main` from `Vagabond-K:develop` 12 hours ago

Conversation 27 Commits 14 Checks 2 Files changed 12 +53 -25

Vagabond-K commented 4 days ago

Description

To ensure compatibility with ESP32-S and ESP32-C series—where TX/RX channel numbers must be explicitly defined—I have modified the constructors for all classes depending on `nanoFramework.Hardware.Esp32.Rmt` to include channel configuration parameters.

Motivation and Context

This change is necessary for users working with newer ESP32 variants (S/C series) where RMT channel assignment is mandatory.

How Has This Been Tested?

I have verified these changes on ESP32-S3 and ESP32-C3.

Screenshots

Reviewers

- Copilot
- josesimoes

Assignees

No one assigned

Labels

Type: enhancement

Projects

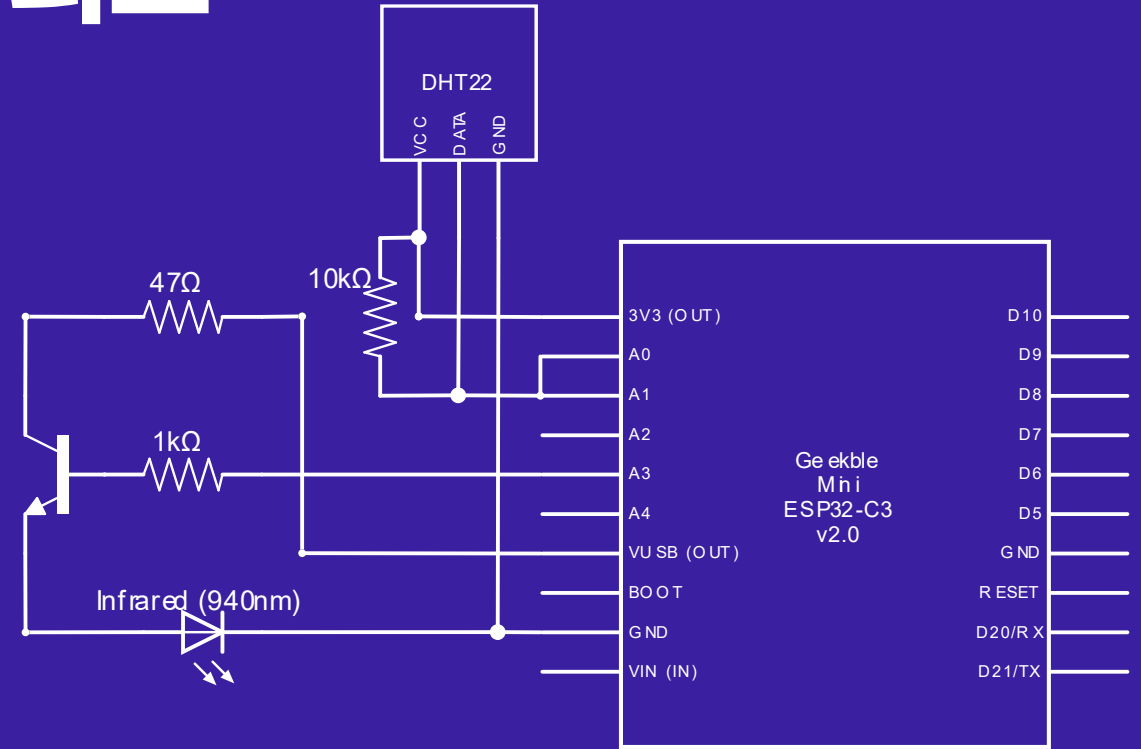
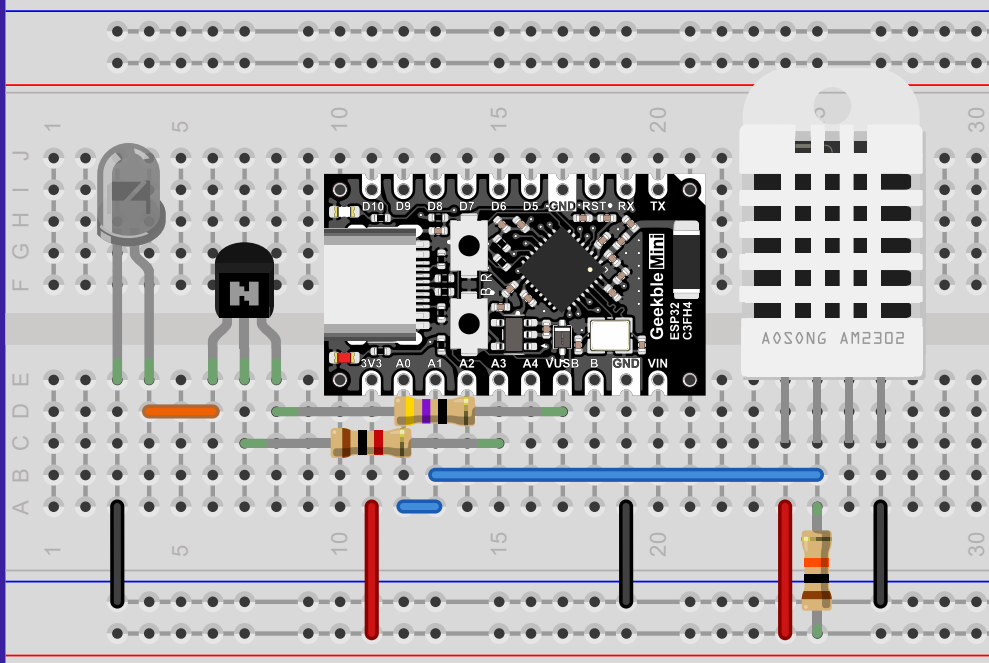
None yet

Milestone

No milestone

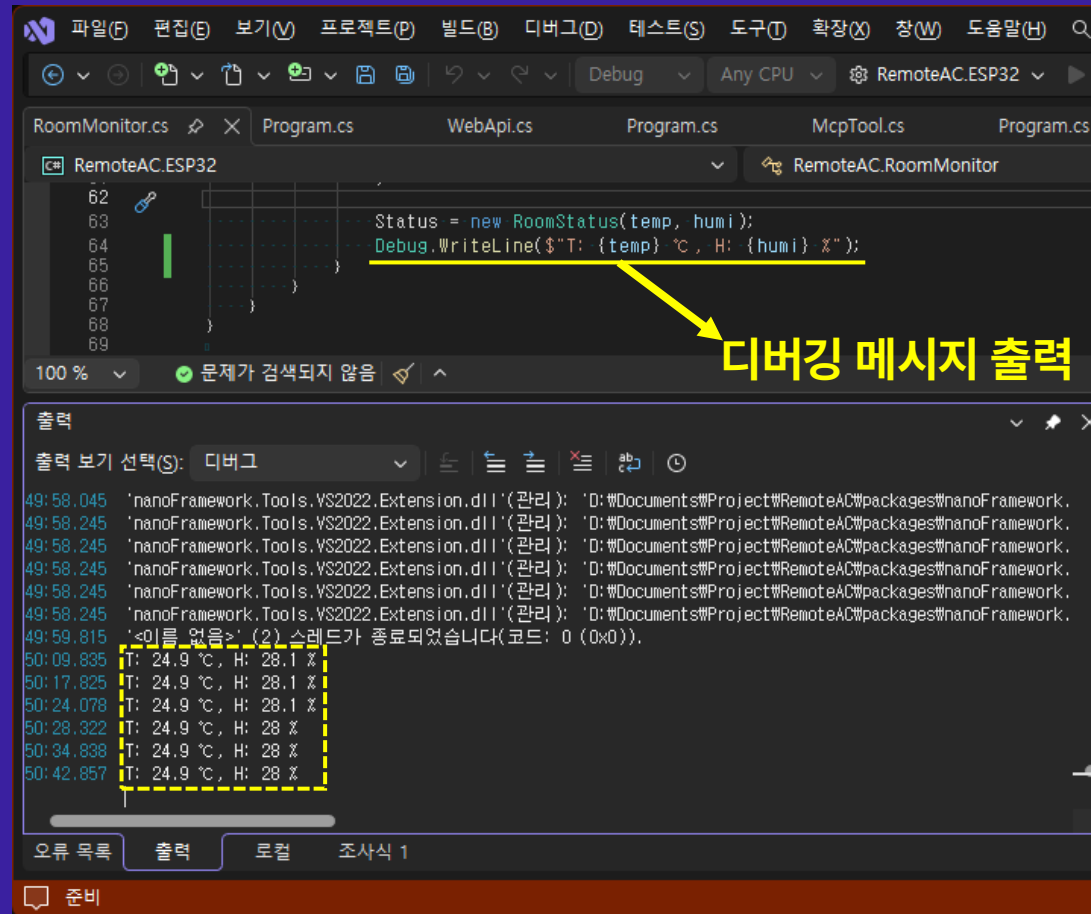
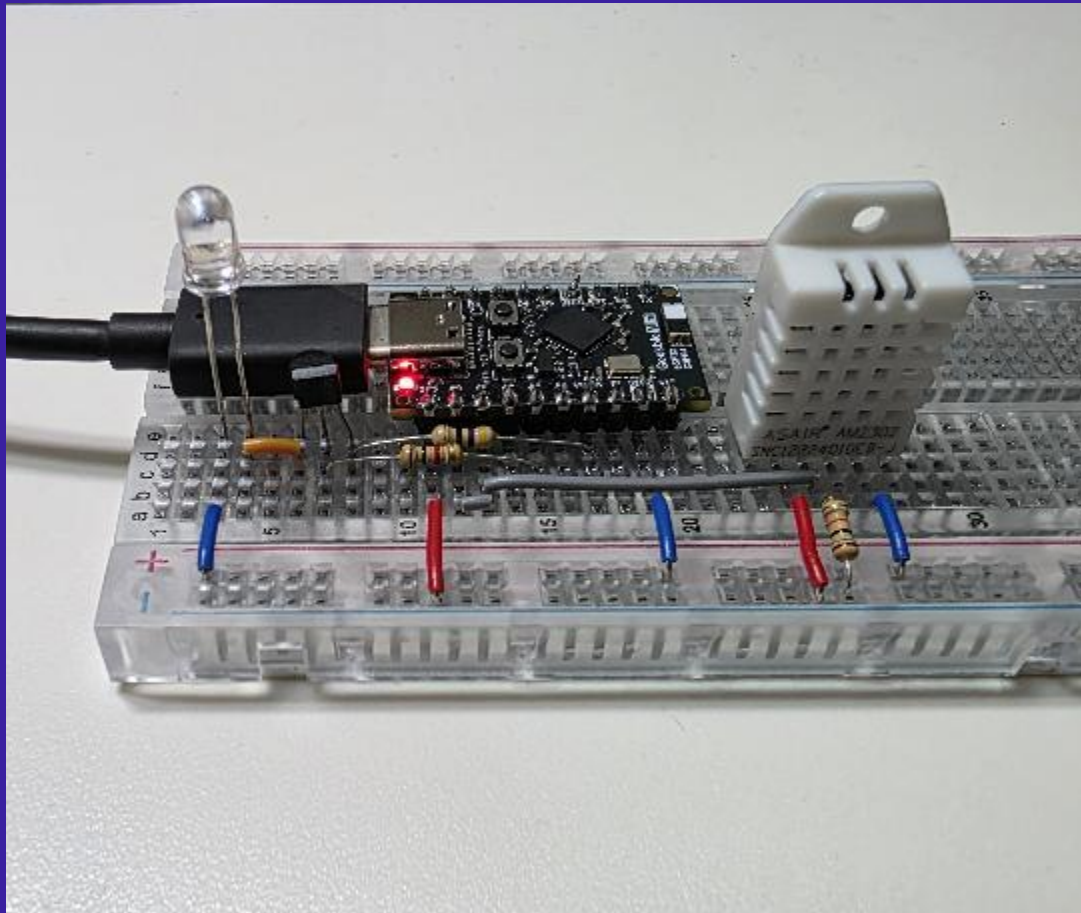
Development

온/습도 센서 포함 최종 회로

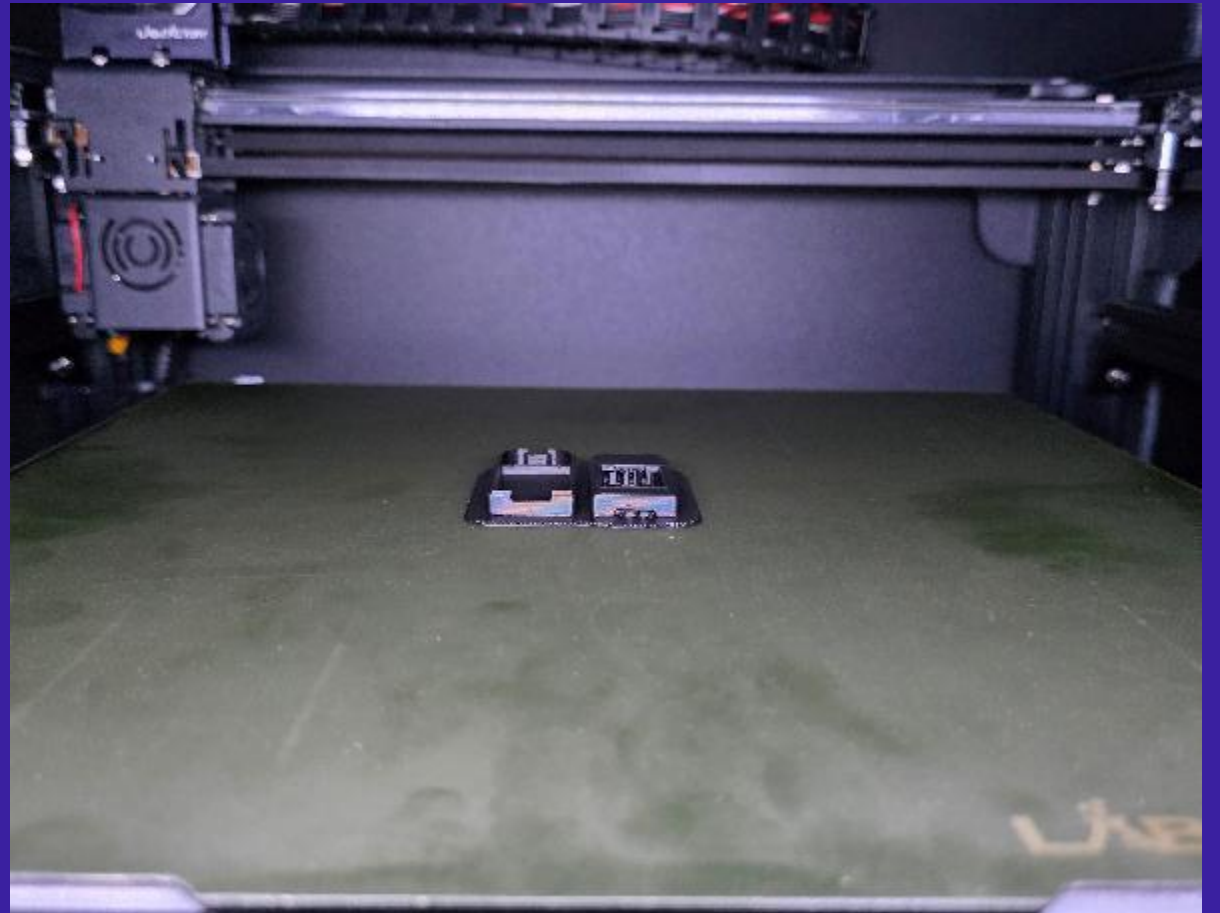
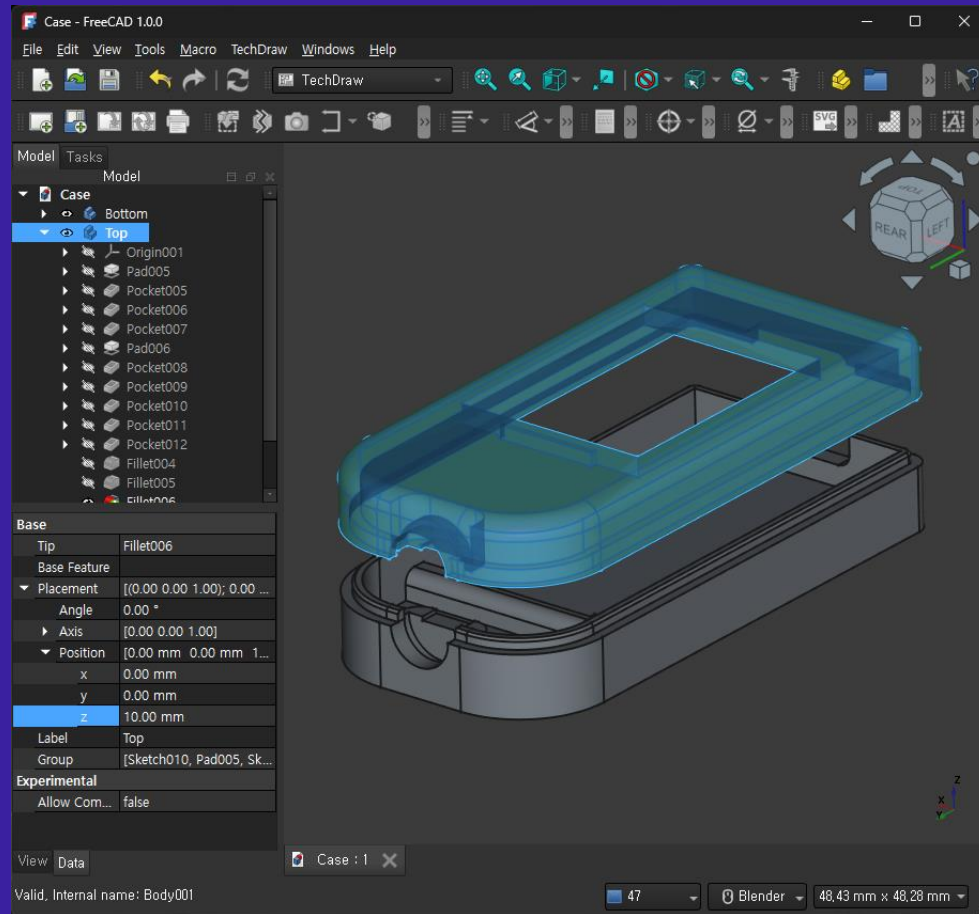


- DHT22의 Data핀과 A0, A1 핀을 연결하여 1-Wire 통신 수행

온/습도 수집 테스트



케이스 설계 및 3D 프린터 출력



부품 연결 및 최종 조립



Web API 기능 추가



MCU 메인 프로그램 코드

```
public class Program
{
    public static AirconRemoteController AirconRemoteController { get; private set; }
    public static RoomMonitor RoomMonitor { get; private set; }

    public static void Main()
    {
        AirconRemoteController = new(channel: 0, pinNumber: 3);
        RoomMonitor = new(channel: 2, pinEcho: 0, pinTrigger: 1);
        RoomMonitor.Start();

        var connected = WifiNetworkHelper.ConnectDhcp("SSID", "PW", requiresDateTime: true, token: new CancellationTokenSource(60000).Token);
        if (!connected) return;

        McpToolRegistry.DiscoverTools(new Type[] { typeof(McpTool) });
        McpServerController.ServerName = "RemoteAC";
        McpServerController.ServerVersion = "1.0.0";
        McpServerController.Instructions = "작업실의 온습도 상태를 가져오거나 에어컨을 제어하는 기능을 제공합니다.";

        using var server = new WebServer(80, HttpProtocol.Http, new Type[] { typeof(WebApi), typeof(McpServerController) });
        server.Start();
        Thread.Sleep(Timeout.Infinite);
    }
}
```

참고: AP 모드 기반 Wi-Fi 설정 페이지 예제

The screenshot shows a web browser window with the address bar displaying `https://docs.nanoframework.net/samplesdetails/WiFiAP/README.html`. The browser's address bar includes navigation icons (back, forward, home, search) and a search bar with the text "Search". The page header features the NanoFramework logo and navigation links: "API Reference", "IoT.Device", "Contributing", "GitHub", and "Discord".

The main content area is titled "Samples / WiFiAP" and features a section for the "Wifi Soft AP sample". The section includes a title with a chili pepper icon, a description of the sample's functionality, and instructions on how to use it. The description states: "Shows how to use various APIs related with Wifi Soft AP. Starts a Soft AP (Hot spot) and runs a simple web server to configure the Wireless connection parameters." It also mentions that the sample uses GPIO pin 5 to switch back into the SoftAP configuration mode and provides a URL to connect to the web server: `http://192.168.4.1/`.

On the right side of the page, there is a sidebar titled "IN THIS ARTICLE" containing links to various sections: "Hardware requirements", "Related topics", "Reference", "Build the sample", "Run the sample", "Deploying the sample", and "Deploying and running the sample".

On the left side of the page, there is a sidebar titled "Filter by title" containing a list of sample names: "ToStringTest", "TouchESP32", "UdpClient", "UnitTest", "UsbClient", "Webserver", "WebSockets", "Wifi", and "WiFiAP". The "WiFiAP" item is currently selected and highlighted.

상태 확인용 Web API 구현

```
public class WebApi
{
    [Route("api/room/status")]
    [Method("GET")]
    public void GetRoomStatus(WebServerEventArgs e) => OutputAsJson(e.Context.Response, Program.RoomMonitor.Status);

    [Route("api/ac/status")]
    [Method("GET")]
    public void GetAirconStatus(WebServerEventArgs e) => OutputAsJson(e.Context.Response, Program.AirconRemoteController.Status);

    [Route("api/ac/set")]
    [Method("POST")]
    public void SendAirconCommand(WebServerEventArgs e)
    {
        var parameters = ParseParams(e.Context.Request.InputStream);

        ...
    }

    ...
}
```

**ASP.NET Core와 유사하나,
경량화로 인해 없는 기능이 많음**

Web API의 JSON 응답 및 파라미터 파싱

```
static void OutputAsJson(HttpListenerResponse response, object obj)
{
    response.ContentType = "application/json";
    WebServer.OutputStream(response, JsonConvert.SerializeObject(obj));
}
```

**객체를 직접 JSON 직렬화 해야 함
그나마 JsonConvert는 지원함**

```
static Hashtable ParseParams(Stream inputStream)
{
    byte[] buffer = new byte[inputStream.Length];
    inputStream.Read(buffer, 0, (int)inputStream.Length);

    string[] parameters = Encoding.UTF8.GetString(buffer, 0, buffer.Length).Split('&');

    Hashtable hashtable = new();
    for (int i = 0; i < parameters.Length; i++)
    {
        string[] tokens = parameters[i].Split('=');
        hashtable[tokens[0]] = tokens[1];
    }

    return hashtable;
}
```

**POST의 urlencoded 파라미터를 직접 파싱해야 함
그나마 GET의 URL 파라미터 파싱 메서드는 있음**

Web API 테스트

The screenshot displays the Postman web application interface. At the top, there is a navigation bar with 'Home', 'Workspaces', and 'Explore' links, a search bar labeled 'Search Postman', and buttons for 'Sign In' and 'Create Account'. Below the navigation bar, a list of recent requests is shown, including 'GET http://192.168.0.171/api/rc', 'GET http://192.168.0.171/api/ac', and 'POST http://192.168.0.171/api/'. The main workspace shows a selected POST request to 'http://192.168.0.171/api/ac/set'. The request body is configured with 'x-www-form-urlencoded' encoding. A table lists the form data with keys 'run', 'mode', 'temp', and 'speed', each with a checked checkbox and a corresponding value. The 'run' value is 'false', 'mode' is 'Auto', 'temp' is '25', and 'speed' is 'Auto'. The response section shows a '200 OK' status with a response time of '420 ms' and a body size of '156 B'. The response body is displayed in 'Pretty' format as a JSON object:

```
{  "IsRun": false,  "OpMode": "Auto",  "FanSpeed": "Auto",  "Temperature": 25}
```

Home Workspaces ▾ Explore

Search Postman

Sign In Create Account

GET http://192.168.0.171/api/rc GET http://192.168.0.171/api/ac POST http://192.168.0.171/api/ + ...

HTTP http://192.168.0.171/api/ac/set Save </>

POST ▾ http://192.168.0.171/api/ac/set Send ▾

Params Auth Headers (8) Body ● Pre-req. Tests Settings ...

x-www-form-urlencoded ▾

	Key	Value	Bulk Edit
☒	run	false	
☒	mode	Auto	
☒	temp	25	
☒	speed	Auto	
	Key	Value	

Body ▾

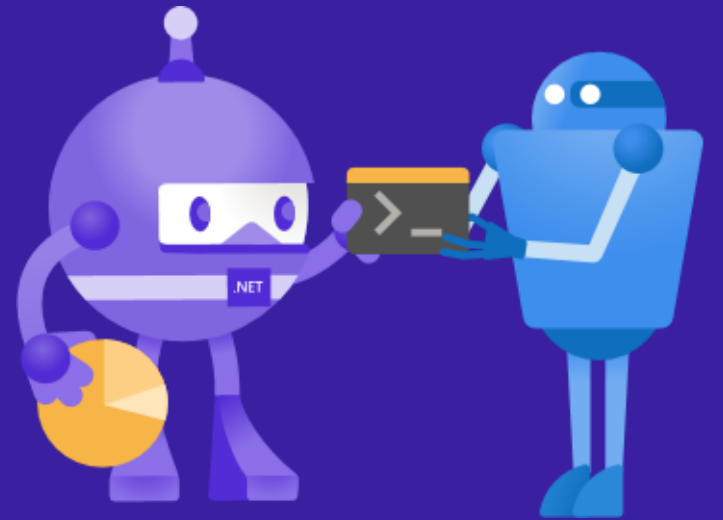
200 OK 420 ms 156 B Save Response ▾

Pretty Raw Preview Visualize JSON ▾

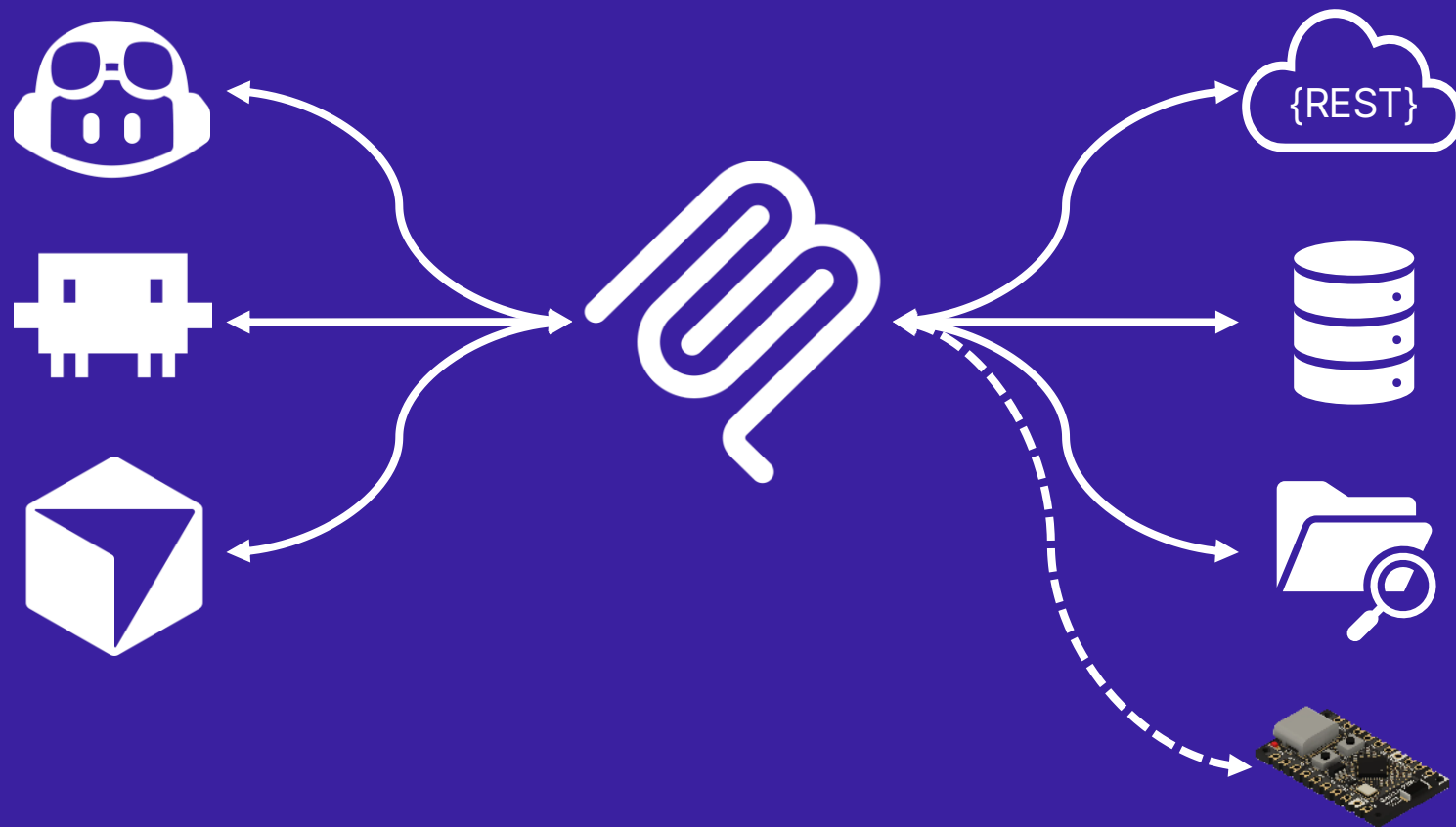
```
1 {
2   "IsRun": false,
3   "OpMode": "Auto",
4   "FanSpeed": "Auto",
5   "Temperature": 25
6 }
```

Console Not connected to a Postman account

MCP 서버 기능 추가



Model Context Protocol



MCP Tool 구현

```
static class McpTool
{
    [McpServerTool("ac_set_temperature", "에어컨의 온도를 섭씨(℃) 기준 정수형으로 설정합니다.", "에어컨 조작 결과를 나타냅니다.")]
    public static string SetTemperature(int temperature)
    {
        var result = "정상적으로 설정했습니다.";
        if (temperature < 17)
            result = "설정 온도는 17℃ 보다 낮게 설정할 수 없습니다. 따라서 가장 낮은 설정 온도인 17℃로 설정했습니다.";
        else if (temperature > 30)
            result = "설정 온도는 30℃ 보다 높게 설정할 수 없습니다. 따라서 가장 높은 설정 온도인 30℃로 설정했습니다.";
        Program.AirconRemoteController.Temperature = (int)Math.Clamp(temperature, 17, 30);
        return result;
    }

    [McpServerTool("ac_turn_on", "에어컨을 켭니다.")]
    public static void TurnOn() => Program.AirconRemoteController.IsRun = true;

    [McpServerTool("ac_turn_off", "에어컨을 끕니다.")]
    public static void TurnOff() => Program.AirconRemoteController.IsRun = false;

    ...
}
```

Tool 이름 → "ac_set_temperature"

Tool 설명 → "에어컨의 온도를 섭씨(℃) 기준 정수형으로 설정합니다."

결과 출력에 대한 설명 → "에어컨 조작 결과를 나타냅니다."

구조화된 MCP Tool 출력 정의

```
static class McpTool
{
    ...
    [McpServerTool("ac_get_status", "에어컨 상태를 가져옵니다.", "에어컨 상태 정보들을 나타냅니다.")]
    public static AirconStatus GetAirconStatus() => Program.AirconRemoteController.Status;
    ...
}
```

```
public class AirconStatus
{
    ...
    public bool IsRun
    {
        [Description("에어컨의 작동 여부를 나타냅니다. true면 작동 중이고, false면 정지 상태입니다.")]
        get;
    }
    ...
}
```

구조화된 결과 출력의 경우 Description 특성은 속성의 getter에 명시해야 함



[Description("에어컨의 작동 여부를 나타냅니다. true면 작동 중이고, false면 정지 상태입니다.")]
get;

VS Code AI 채팅에서 써보자! ...오류 발생?

The screenshot shows the Visual Studio Code interface with a dark theme. The left sidebar contains the Explorer view with a folder named 'MCPTEST' and a file named 'mcp.json'. The main editor area displays the 'mcp.json' file with the following content:

```
.vscode > {} mcp.json > {} servers
1 {
2   "servers": {
3     "RemoteAC": {
4       "url": "http://192.168.0.171/mcp",
5       "type": "http"
6     }
7   },
8   "inputs": []
9 }
```

Below the editor, the Output view shows logs for 'MCP: RemoteAC'. The logs indicate that the server is starting successfully, but an error occurred at 16:45:00.266:

```
2026-03-08 16:44:59.666 [info] 서버 RemoteAC 시작하는 중
2026-03-08 16:44:59.667 [info] 연결 상태: 시작
2026-03-08 16:44:59.668 [info] Starting server from LocalProcess extension host
2026-03-08 16:44:59.668 [info] 연결 상태: 실행 중
2026-03-08 16:45:00.266 [error] Error: MPC -32602: Unsupported protocol version
at vscode-file://vscode-app/c:/Users/bruce7615/AppData/Local/Programs/
Microsoft%20VS%20Code/61b3d0ab13/resources/app/out/vs/workbench/workbench.desktop.main.
```

On the right side, the AI Chat window is open, showing a 'Build with Agent' section. It includes a tip: 'Tip: Try the Plan agent to research and plan before implementing changes.' Below this, there is a button to '+ {} mcp.json' and a section for '다음에 빌드할 내용 설명' (Explain content to build next).

MCP 응답: 지원되지 않는 프로토콜 버전

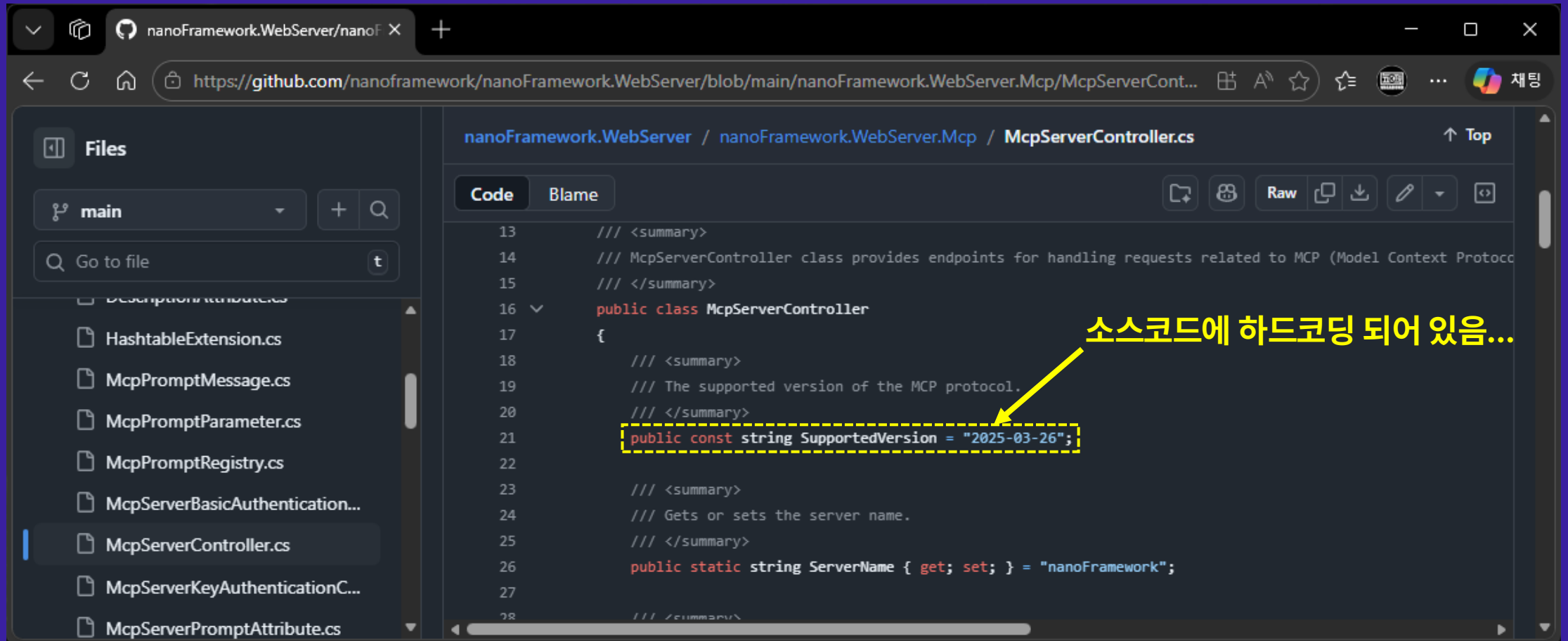
The image shows a Wireshark packet capture of an HTTP request and response. The packet list shows three packets: a POST request (No. 463), an OK response (No. 476), and a GET request (No. 484). The selected packet (No. 476) is an HTTP 200 OK response with a Content-Type of application/json. The packet details pane shows the JSON object structure, including the 'error' field with a code of -32602 and a message of 'Unsupported protocol version'. The packet bytes pane shows the raw data of the response.

No.	Time	Source	Destination	Protocol	Length	Info
463	30.053486900	192.168.0.210	192.168.0.171	HTTP/1.1	772	POST /mcp HTTP/1.1, JSON (application/json)
476	30.438512000	192.168.0.171	192.168.0.210	HTTP/1.1	205	HTTP/1.1 200 OK, JSON (application/json)
484	30.444967400	192.168.0.210	192.168.0.171	HTTP	261	GET /mcp HTTP/1.1

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Unsupported protocol version",
    "data": {
      "supported": [
        "2025-03-26"
      ],
      "requested": "2025-11-25"
    }
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Unsupported protocol version",
    "data": {
      "supported": [
        "2025-03-26"
      ],
      "requested": "2025-11-25"
    }
  }
}
```

소스코드 확인

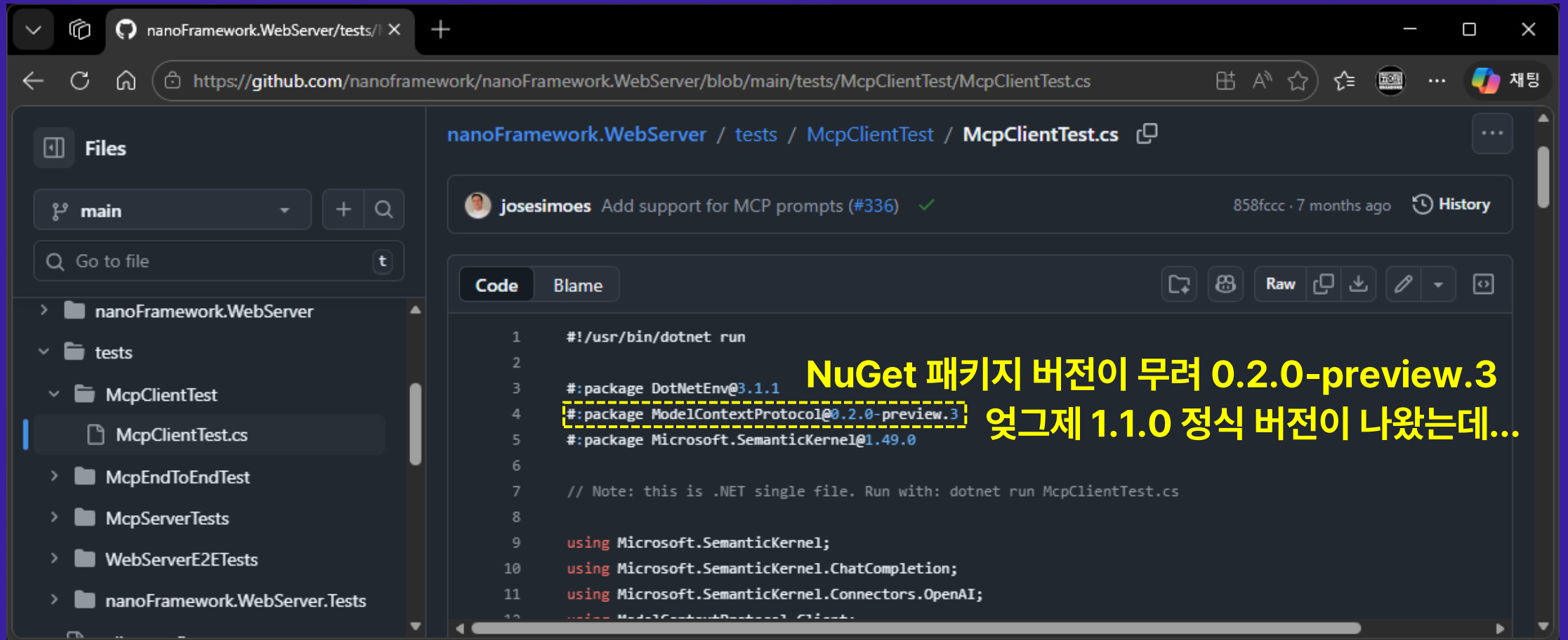


The screenshot shows a web browser displaying the GitHub repository for nanoFramework.WebServer. The file path is nanoFramework.WebServer / nanoFramework.WebServer.Mcp / McpServerController.cs. The code is shown in a dark theme. The left sidebar shows the file explorer with the following files: HashtableExtension.cs, McpPromptMessage.cs, McpPromptParameter.cs, McpPromptRegistry.cs, McpServerBasicAuthentication..., McpServerController.cs (selected), McpServerKeyAuthenticationC..., and McpServerPromptAttribute.cs. The main area shows the code for McpServerController.cs. The code is as follows:

```
13    /// <summary>
14    /// McpServerController class provides endpoints for handling requests related to MCP (Model Context Protocol)
15    /// </summary>
16    public class McpServerController
17    {
18        /// <summary>
19        /// The supported version of the MCP protocol.
20        /// </summary>
21        public const string SupportedVersion = "2025-03-26";
22
23        /// <summary>
24        /// Gets or sets the server name.
25        /// </summary>
26        public static string ServerName { get; set; } = "nanoFramework";
27
28        /// <summary>
```

A yellow arrow points to the line `public const string SupportedVersion = "2025-03-26";` with the text "소스코드에 하드코딩 되어 있음..." (Hardcoded in the source code...).

공식 예제 테스트 → 잘 동작함



The screenshot shows a web browser displaying a GitHub repository for `nanoFramework.WebServer`. The file path is `tests/McpClientTest/McpClientTest.cs`. The file is a C# script for testing the MCP client. A yellow dashed box highlights the NuGet package versions in the code, with a yellow text annotation pointing to them.

Annotation: NuGet 패키지 버전이 무려 0.2.0-preview.3 엇그제 1.1.0 정식 버전이 나왔는데...

```
1  #!/usr/bin/dotnet run
2
3  #:package DotNetEnv@3.1.1
4  #:package ModelContextProtocol@0.2.0-preview.3
5  #:package Microsoft.SemanticKernel@1.49.0
6
7  // Note: this is .NET single file. Run with: dotnet run McpClientTest.cs
8
9  using Microsoft.SemanticKernel;
10 using Microsoft.SemanticKernel.ChatCompletion;
11 using Microsoft.SemanticKernel.Connectors.OpenAI;
12 using ModelContextProtocol.Client;
```

MCP 패키지 버전 최신화 시도

```
var mcpClient = await McpClient.CreateAsync(new HttpClientTransport(new HttpClientTransportOptions()
{
    Endpoint = new Uri("http://192.168.0.171/mcp"),
}), new McpClientOptions { ProtocolVersion = "2025-03-26" });

var tools = await mcpClient.ListToolsAsync();

var agent = new OpenAIClient(
    new ApiKeyCredential("local"), new OpenAIClientOptions{ Endpoint = new Uri("http://localhost:11434/v1") })
    .GetChatClient("llama3.1:8b")
    .AsAIAgent(name: "MyAgent", tools: [.. tools.Cast<AITool>()]);

var session = await agent.CreateSessionAsync();

string? userInput;
Console.Write("User: ");
while ((userInput = Console.ReadLine()) is not null)
{
    var result = await agent.RunAsync(userInput, session);
    Console.WriteLine("Agent: " + result.Text);
    Console.Write("User: ");
}
```

ModelContextProtocol v1.1.0 적용
Microsoft Agent Framework 적용

예외 발생: 유효한 OutputSchema가 아닙니다.

The screenshot shows the Visual Studio IDE with a C# program named `Program.cs` in the `McpClientTest` project. The code is in debug mode, and an exception has occurred at line 12: `var tools = await mcpClient.ListToolsAsync();`. The exception is a `System.ArgumentException` with the message: "The specified document is not a valid MCP tool output JSON schema. (Parameter 'OutputSchema')".

The output window shows the following log entries:

```
28:52.476 'McpClientTest.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\10.0.3\System.Private.CoreLib.dll'
28:52.476 'McpClientTest.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\10.0.3\System.Private.CoreLib.dll'
28:52.476 'McpClientTest.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\10.0.3\System.Private.CoreLib.dll'
28:52.476 'McpClientTest.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\10.0.3\System.Private.CoreLib.dll'
28:52.881 'McpClientTest.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\10.0.3\System.Private.CoreLib.dll'
28:52.881 'McpClientTest.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\10.0.3\System.Private.CoreLib.dll'
28:52.881 'McpClientTest.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\10.0.3\System.Private.CoreLib.dll'
28:52.881 'McpClientTest.exe' (CoreCLR: clrhost): 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\10.0.3\System.Private.CoreLib.dll'
28:53.482 예외 발생: 'System.ArgumentException' (System.Private.CoreLib.dll)
28:53.482 'System.ArgumentException' 형식의 예외가 System.Private.CoreLib.dll에서 발생했지만 사용자 코드에서 처리되지 않았습니다.
28:53.482 The specified document is not a valid MCP tool output JSON schema.
```

The exception settings dialog is open, showing the following options:

- ☐ 이 예외 형식이 throw되면 중단
- ☒ 이 예외 형식을 사용자가 처리하지 않은 경우 중단

다음에서 발생한 경우 제외:

- ☐ McpClientTest.dll

예외 설정 열기 | 조건 편집

void로 선언된 메서드의 출력이 string이다?

The image shows a Wireshark packet capture of an HTTP POST request to `/mcp` from `192.168.0.171` to `192.168.0.210`. The selected packet (219) is an HTTP/1.1 200 OK response with a JSON body. The JSON object contains a `jsonrpc` field with value `2.0`, an `id` field with value `2`, and a `result` field which is an object containing a `tools` array. The `tools` array has one element with `name` `ac_turn_on` and `description` `에어컨을 켭니다.`. The `outputSchema` field is an object with `type` `string`.

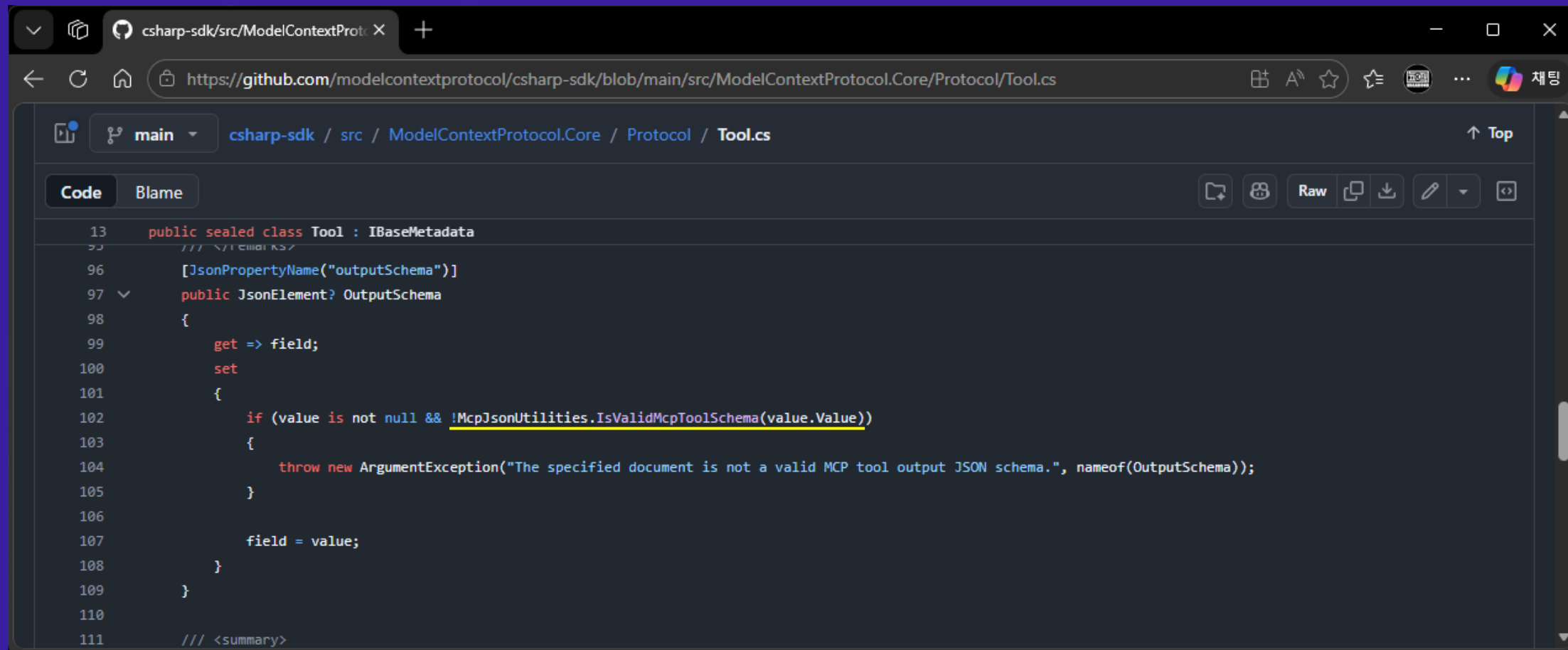
```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tools": [
      {
        "name": "ac_turn_on",
        "description": "에어컨을 켭니다.",
        "outputSchema": { "type": "string" }
      }
    ]
  },
  "nextCursor": null
}
```

```
[McpServerTool("ac_turn_on", "에어컨을 켭니다.")]
public static void TurnOn()
=> Program.AirconRemoteController.IsRun = true;
```

string, int, bool 등으로 수정해도 동일한 예외 발생

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tools": [
      {
        "name": "ac_turn_on",
        "description": "에어컨을 켭니다.",
        "outputSchema": { "type": "string" }
      }
    ]
  },
  "nextCursor": null
}
```

MCP C# SDK 소스코드 분석



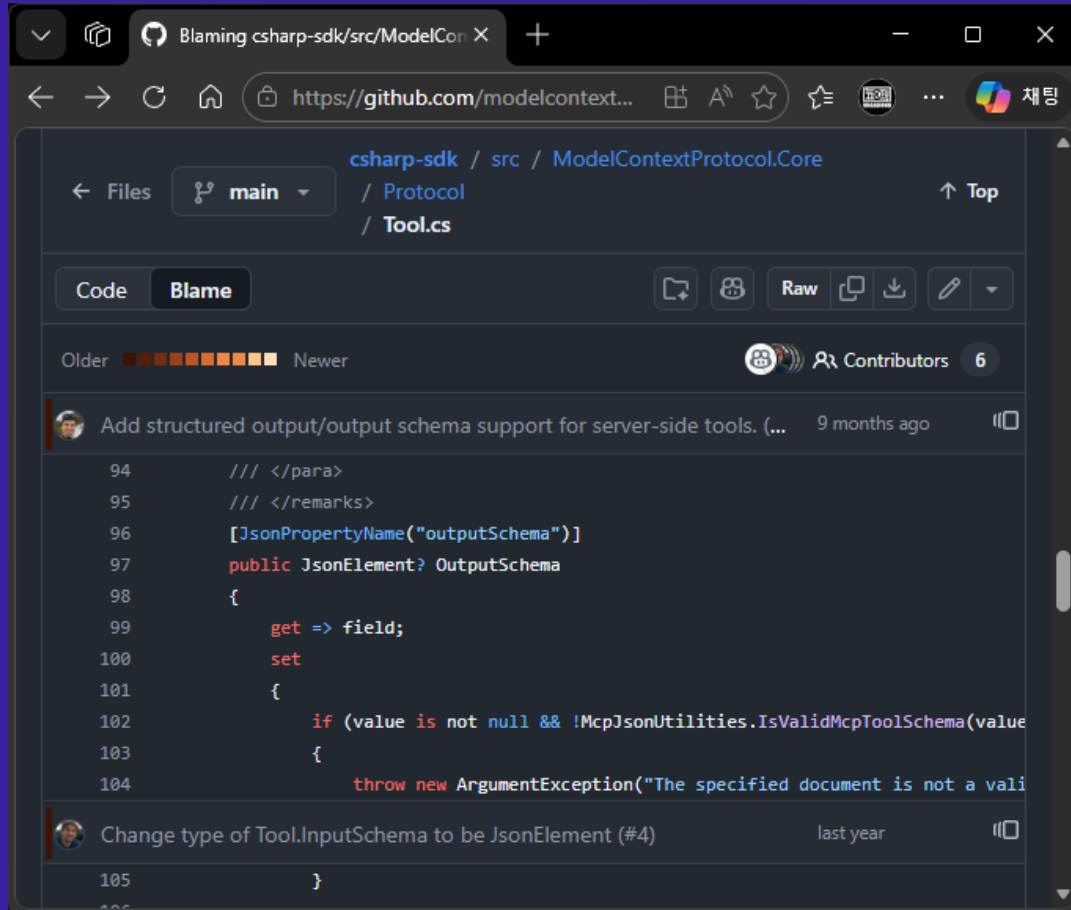
The screenshot displays the GitHub web interface for the file `Tool.cs` within the `ModelContextProtocol.Core` directory of the `csharp-sdk` repository. The browser's address bar shows the URL `https://github.com/modelcontextprotocol/csharp-sdk/blob/main/src/ModelContextProtocol.Core/Protocol/Tool.cs`. The file explorer at the top indicates the current path is `main / csharp-sdk / src / ModelContextProtocol.Core / Protocol / Tool.cs`. The code editor shows the following C# code:

```
13 public sealed class Tool : IBaseMetadata
14 {
15     /// <remarks>
16     [JsonPropertyName("outputSchema")]
17     public JsonElement? OutputSchema
18     {
19         get => field;
20         set
21         {
22             if (value is not null && !McpJsonUtilities.IsValidMcpToolSchema(value.Value))
23             {
24                 throw new ArgumentException("The specified document is not a valid MCP tool output JSON schema.", nameof(OutputSchema));
25             }
26             field = value;
27         }
28     }
29 }
30 /// <summary>
```

OutputSchema는 object여야만 한다!

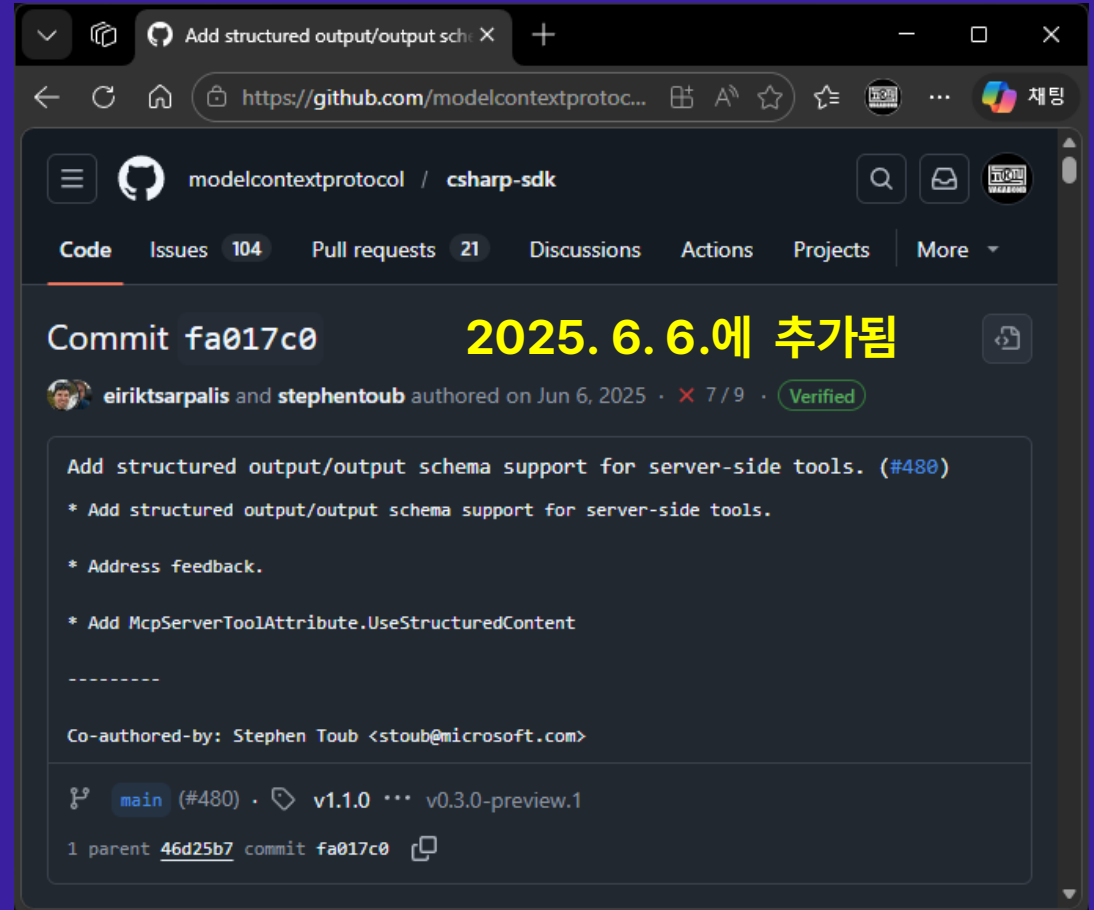
```
63  internal static bool IsValidMcpToolSchema(JsonElement element)
64  {
65      if (element.ValueKind is not JsonValueKind.Object)
66      {
67          return false;
68      }
69
70      foreach (JsonProperty property in element.EnumerateObject())
71      {
72          if (property.NameEquals("type"))
73          {
74              if (property.Value.ValueKind is not JsonValueKind.String ||
75                  !property.Value.ValueEquals("object"))
76              {
77                  return false;
78              }
79
80              return true; // No need to check other properties
81          }
82      }
83
84      return false; // No type keyword found.
85  }
```

OutputSchema 추가 이력 조회



This screenshot shows the GitHub interface for the file `Tool.cs` in the `ModelContextProtocol.Core` directory of the `csharp-sdk` repository. The file is on the `main` branch. The code shows the definition of the `OutputSchema` property, which is a `JsonElement?`. The code includes comments in Korean and a validation check for the schema.

```
94    /// </para>
95    /// </remarks>
96    [JsonPropertyName("outputSchema")]
97    public JsonElement? OutputSchema
98    {
99        get => field;
100        set
101        {
102            if (value is not null && !McpJsonUtilities.IsValidMcpToolSchema(value
103            {
104                throw new ArgumentException("The specified document is not a vali
105        }
106    }
```



This screenshot shows the GitHub commit page for the commit `fa017c0` in the `csharp-sdk` repository. The commit was authored by `eiriktsarpalis` and `stephentoub` on June 6, 2025. The commit message is "Add structured output/output schema support for server-side tools. (#480)". The commit includes a list of changes and a co-authored-by line for Stephen Toub.

Commit `fa017c0` **2025. 6. 6.에 추가됨**

`eiriktsarpalis` and `stephentoub` authored on Jun 6, 2025 · 7 / 9 · **Verified**

Add structured output/output schema support for server-side tools. (#480)

- * Add structured output/output schema support for server-side tools.
- * Address feedback.
- * Add `McpServerToolAttribute.UseStructuredContent`

Co-authored-by: Stephen Toub <stoub@microsoft.com>

`main` (#480) · `v1.1.0` ... `v0.3.0-preview.1`

1 parent `46d25b7` commit `fa017c0`

.NET nanoFramework의 MCP 소스 분석

- .NET nanoFramework의 MCP 서버는 2025. 6. 27.에 추가됨
 - 개발 과정에서 뭔가 혼선이 있었던 것 같음
- Object로 구조화 되지 않은 출력이면, 해당 형식으로 굳이 OutputSchema를 생성
 - `"outputSchema": { "type": "boolean" }`
- Void인 경우도 굳이 string 형식으로 OutputSchema를 생성
 - `"outputSchema": { "type": "string" }`
- **MCP C# SDK에서는 구조화된 출력일 때만 OutputSchema를 생성**
 - **또 .NET nanoFramework 리포지토리에 PR 해볼까?**

PR은 나중에 하고, 일단은 되게 해보자!

```
...

[McpServerTool("ac_turn_on", "에어컨을 켭니다.")]
public static ControlResult TurnOn()
{
    Program.AirconRemoteController.IsRun = true;
    return ControlResult.OK;
}

...

public class ControlResult
{
    public static ControlResult OK { get; } = new ControlResult("정상적으로 설정했습니다.");

    public ControlResult(string result)
    {
        Result = result;
    }

    public string Result
    {
        [Description("에어컨 조작 결과를 나타냅니다.")]
        get;
    }
}
```

모든 MCP 도구 결과를 구조화된 객체로 반환하도록 처리

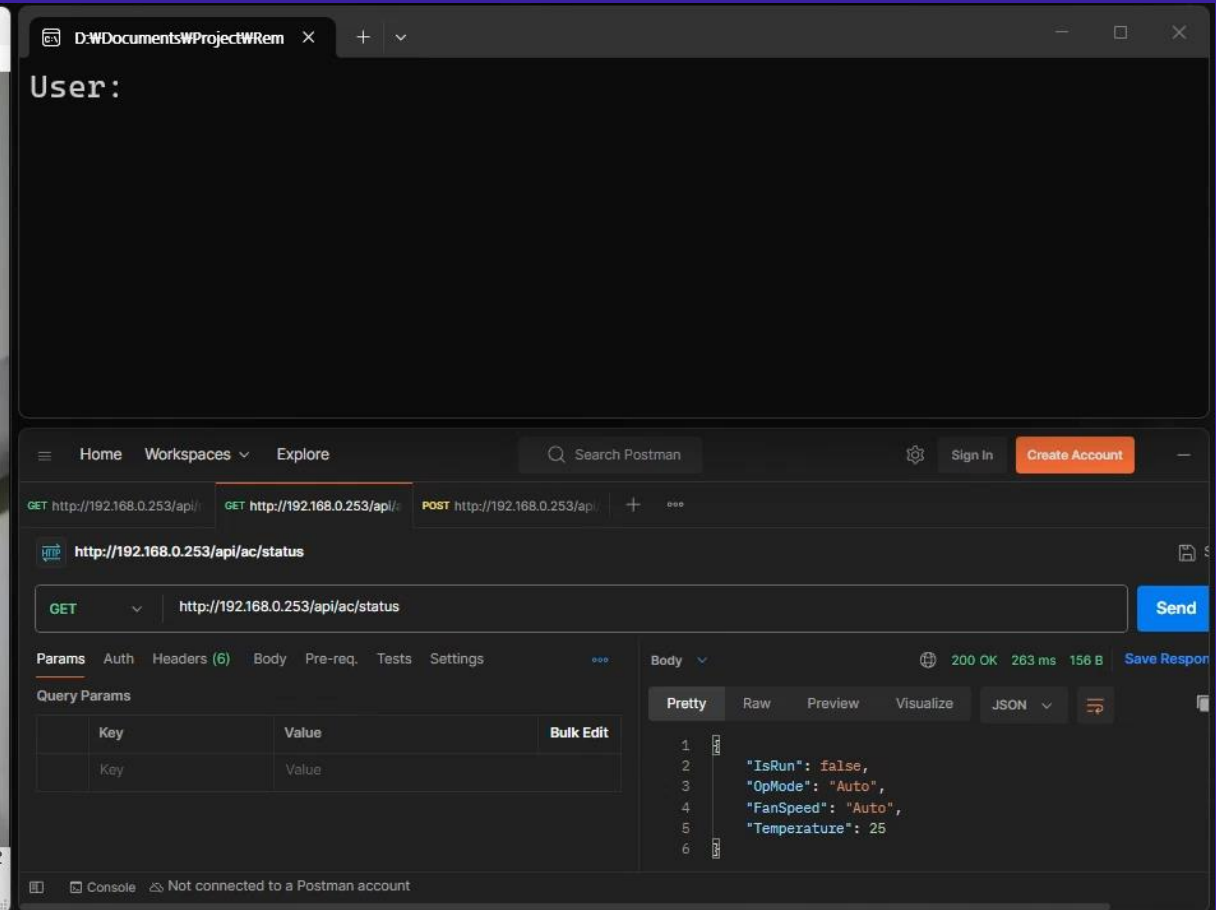
Demonstration



시연 환경



시연 영상



마무리하며...



.NET nanoFramework은 과연 쓸만한가?

- 사용자가 적어서 그런 것인가? 응용 라이브러리 완성도가 부실한 느낌
 - DHT22, MCP 서버 이슈 발견
 - **그렇지만 어떻게든 잘 되긴 된다?**
- 제네릭(IEnumerable<T> 등등)을 쓸 수 없어서 크게 실망함
 - **v2.0 개발이 한창 진행 중**, 제네릭 사용 가능 예정
- 대량 생산 제품에는 그다지... **1회성이나 맞춤형 프로젝트**에 좋을 듯
 - MCU 단가를 낮추면 사양도 낮아지므로 C/C++ 쓸 수밖에 없음
- 저 예산 프로젝트에서 값 비싼 PLC 대신 사용 가능할 것 같은 느낌
 - 저렴한 개발 보드에다가 **PLC 래더 개발자 필요 없이** C#으로 코딩

Thank you...

...But?



발표자 소개 - 김준형 Ph.D.

- 경남 창원의 어느 중소기업에서 20년 가까이 개발자로 근무 중
- 에너지 관리 시스템 개발 및 에너지 기술 연구 과제 수행
- 연구기관 SW 개발 용역 수행 (한국전기연구원, 한국전자기술연구원)
- Microsoft MVP (2025), 닷넷데브 운영진
- 개인 활동: Blog, GitHub, LinkedIn, YouTube
- 닉네임: Vagabond K



www.vagabond-k.com

Thank you!

